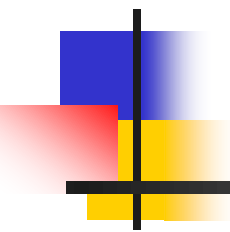
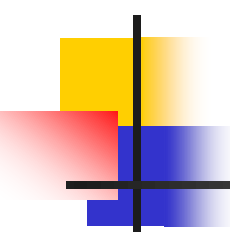


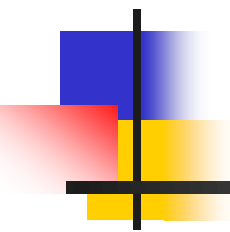
Part Two



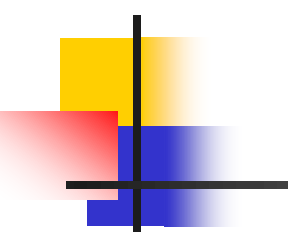
Database Design



Chapter 7



Database Design and E-R Model

- 
-
- Design Process
 - Modeling
 - Constraints
 - E-R Diagram
 - Design Issues
 - Weak Entity Sets
 - Extended E-R Features
 - Design of the Bank Database
 - Reduction to Relation Schemas
 - Database Design
 - UML

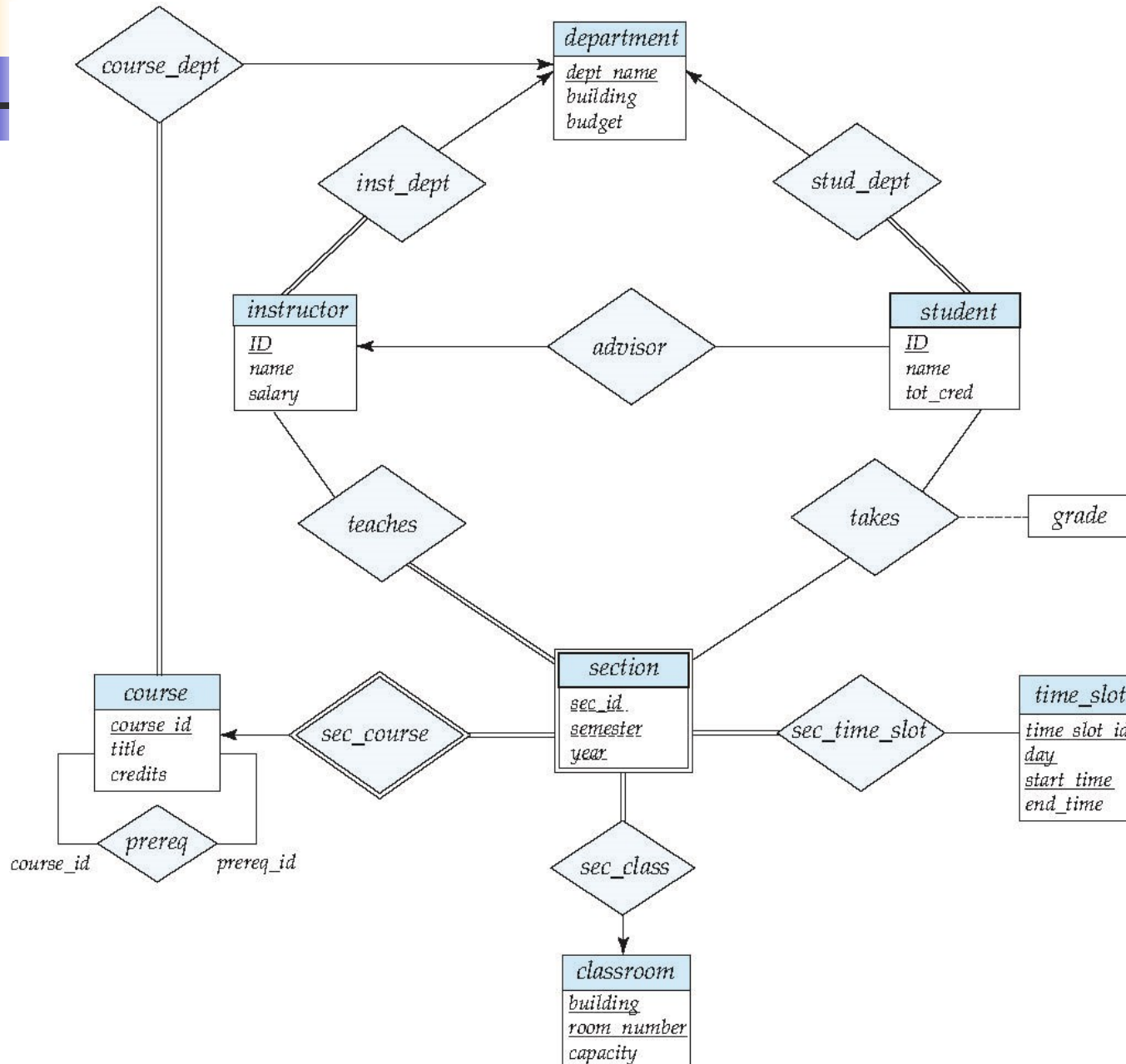
Parts in Chapter 7

- Part 1. DB/DBS/DBAS design process (§ 7.1/7.10 + Supplementary)
 - DB design phases (§ 7.1)
 requirement analysis, conceptual design, logical design,
 physical design
 - DBAS life-cycle model (生命周期模型)
- Part 2. The Entity-Relationship Model
 - *basic* E-R model
 - modeling elements (§ 7.2.1-7.2.3): entity sets, relationship sets, attributes
 - constraints (§ 7.3.1-7.3.3): mapping cardinality, participation constraint, keys
 - weak entity sets(§ 7.5.6)
 - removing redundant attributes in entity sets (§ 7.4)
 - E-R diagram(§ 7.5.1-7.5.5) and alternative notations(§ 7.9)

Parts in Chapter 7

- *extended* E-R Features (§ 7.8)
 - (§ 7.8.1-7.8.4) OO features in E-R model, i.e. specialization, generalization, attributes inheritance, constraints on generalization
 - (§ 7.8.5) aggregation: relationship among relationships
- Part 3. Reduction to Relational Schemas(§ 7.6, 7.8.6)
 - mapping elements in E-R model to that in relational models, i.e. conceptual schema → initial logical schema
- Part 4. E-R design issues (§ 7.7)
 - when applying E-R model to model the objects in real worlds, some issues (considerations and principles) should be addressed, to guarantee DBS effectiveness and efficiency for data

E-R Diagram for a University Enterprise



§ 7.1 Overview of the Design Process

- Database design consists of two sequential phases
 - analyzing of user requirements
 - *what data* should be stored in the database
 - what operations/transaction, such as *insert*, *delete*, *update* and *retrieve* are needed to conducted on these data
 - designing of DB schemas , in accordance with the *three-level of data abstract* (refer to Fig. 1.1)
 - conceptual design
 - logical design, at the logical level and view level
 - physical design, at the physical level
 - refer to Fig.1.0.1

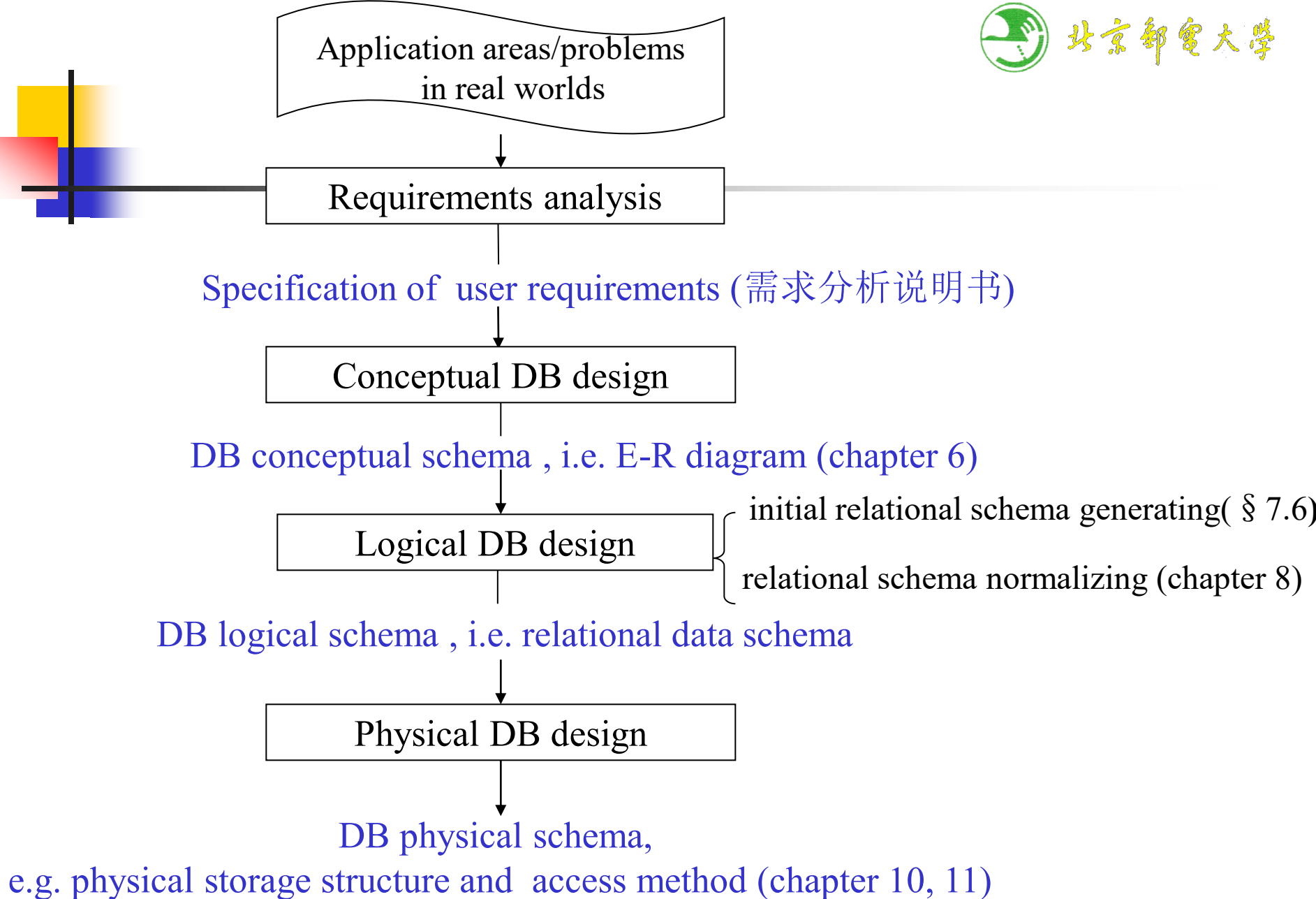
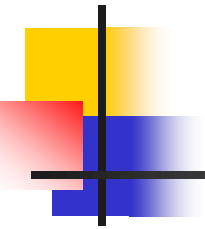
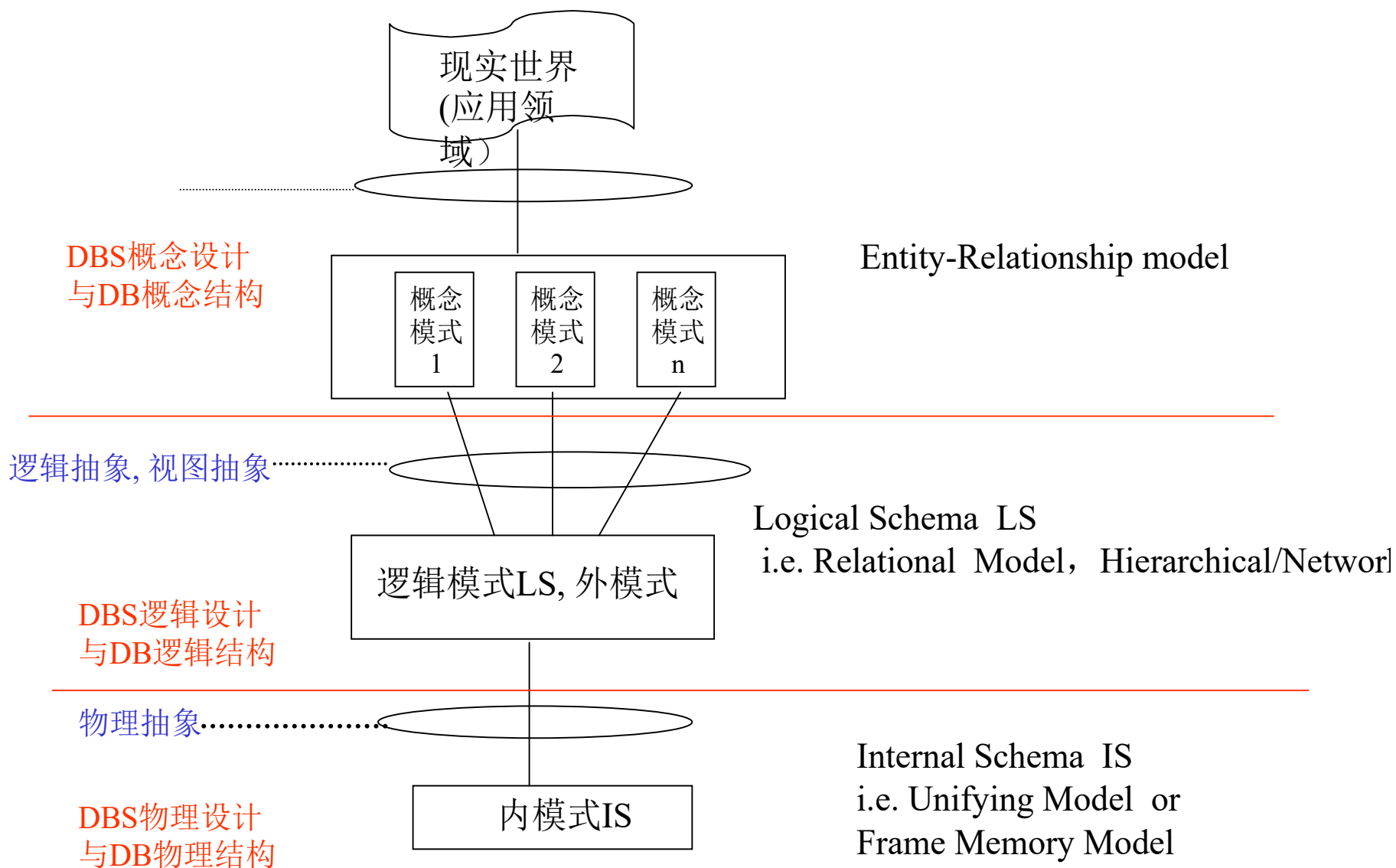


Fig. 7.0.1 DB design phases



DBS设计与设计阶段：数据模型—数据抽象—数据模式

Overview of the Design Process (cont.)

■ 数据库应用系统DBAS设计

- DB, DBMS, users, **application programs**
- refer to Fig. 7.0.2 电信网管系统示意图

■ DBAS设计

- refer to Fig. 7.0.3 DBAS 生命周期模型
- 参照软件工程中软件开发瀑布模型原理，DBAS的生命周期由项目规划、需求分析、系统设计、实现与部署、运行管理与维护等5个基本活动组成
- 根据DBAS的软件组成和各自功能，分为**数据组织与存储设计**、**数据访问与处理设计**、**应用设计**三条设计主线，分别用于设计数据库、数据库事务和应用程序

Overview of the Design Process (cont.)

- 根据数据库系统三级模式结构，DBAS设计阶段分为概念设计、逻辑设计、物理设计三个步骤，每一步设计内容涵盖了三条设计主线

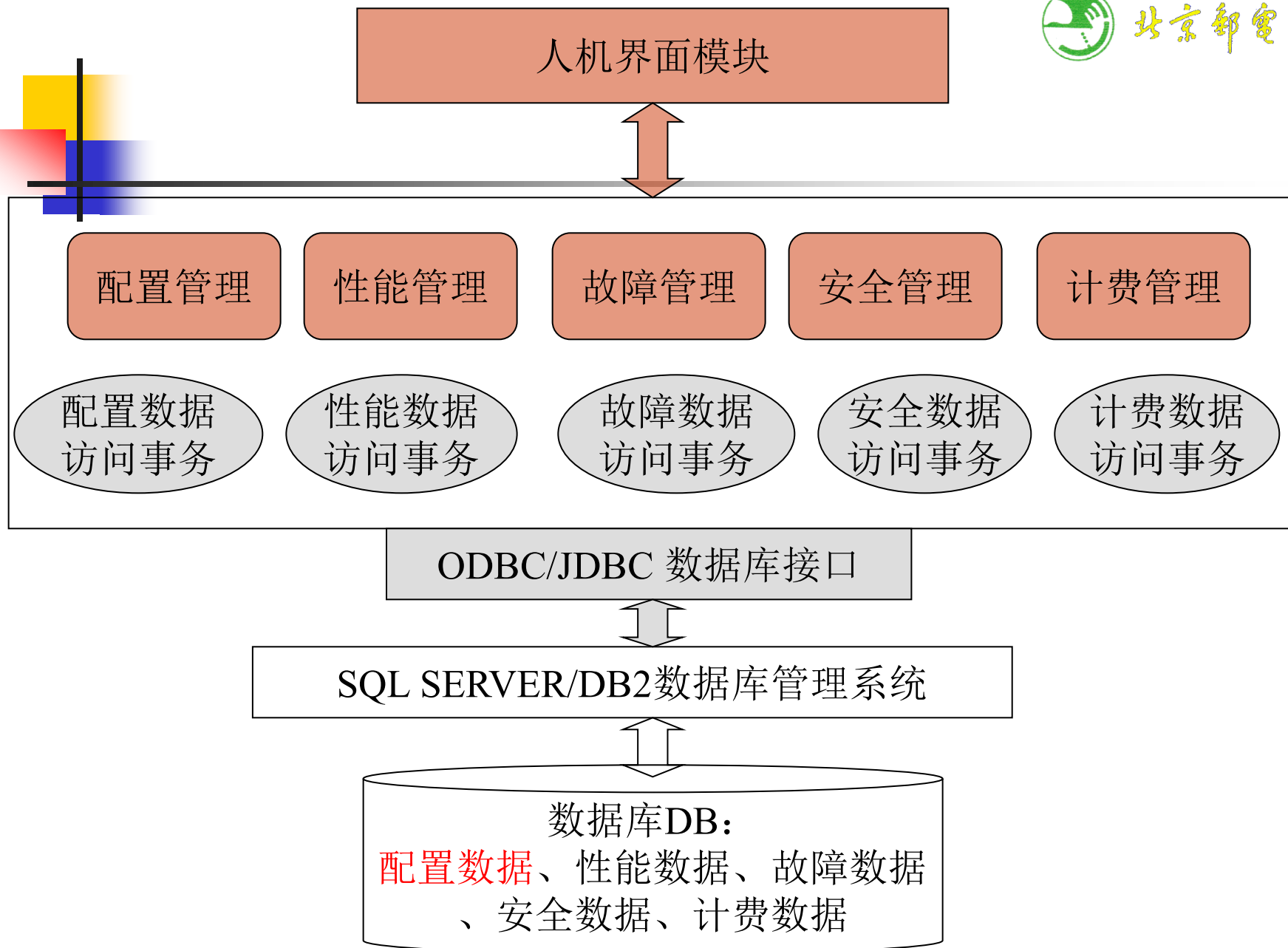


Fig. 7.0.2 电信网管系统示意图

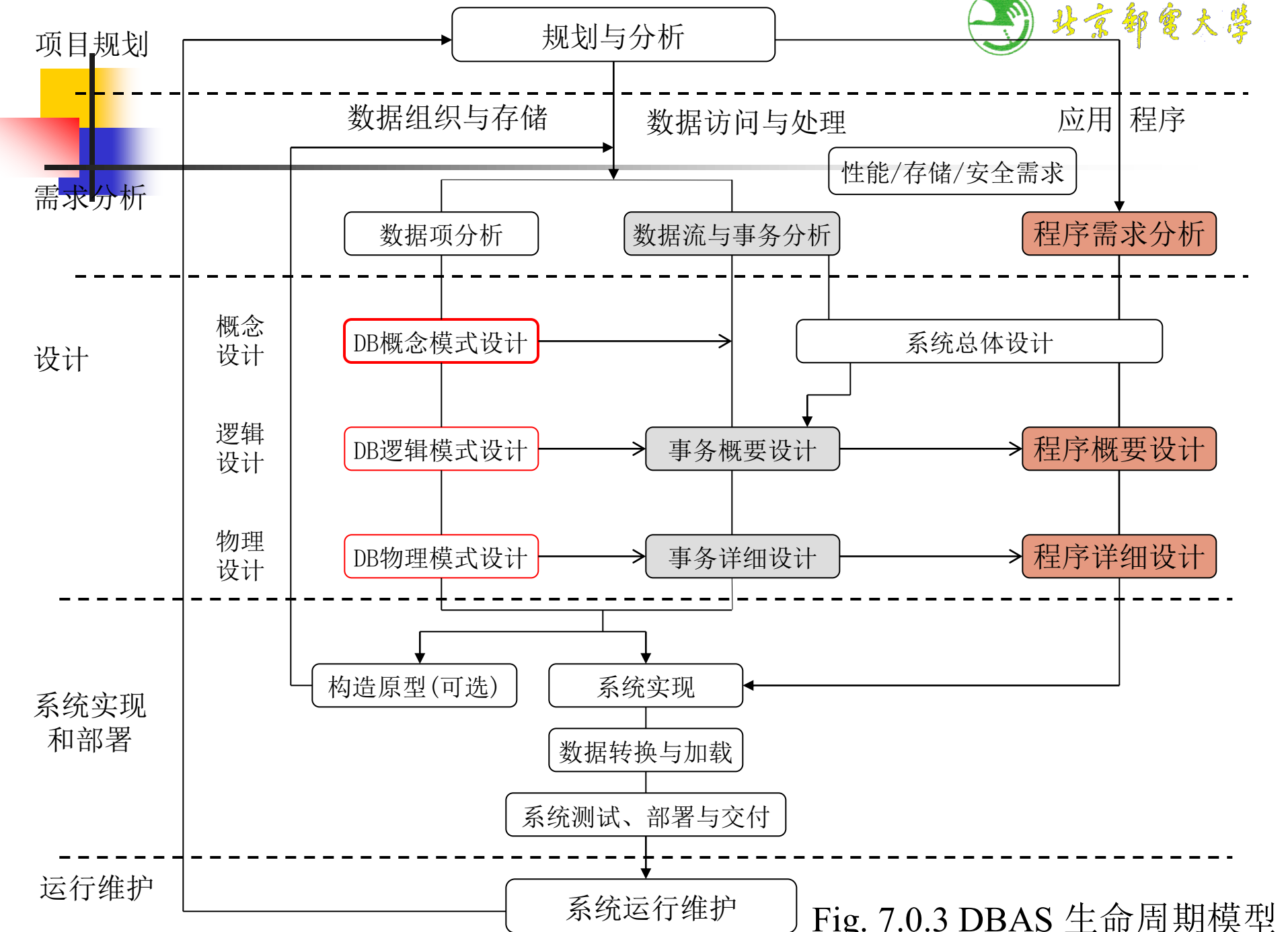


Fig. 7.0.3 DBAS 生命周期模型

中国移动网管支撑系统规范CMOSS2.0——性能指标要求



系统运行基本指标要求如下（仅供参考）：

No	分类	指标	参考值
1	可用性指标	系统可用性	99.9%（7×24）
2		故障平均恢复时间	2 小时
3		故障发生频次	3 次/年
4	应用性能指标	操作响应时延	≤5 秒
5		复杂报表生成时延	≤20 秒
6	系统容量指标	总用户数	按需
7		并发用户数	总用户数×10%
8		存储容量	按需
9		处理能力	按需
10		系统预留容量（存储和处理能力）	≥3 年，年增长≥20%
11		服务器 CPU 峰值利用率	≤70%
12		服务器内存峰值利用率	≤90%
13		有效存储空间利用率	≤80%

7.2 The Entity-Relationship Model

- The ER data model was developed to facilitate database design by allowing specification of an **enterprise schema** that represents the overall logical structure of a database.
- The ER model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the ER model.
- The ER data model employs three basic concepts:
 - entity sets,
 - relationship sets,
 - attributes.
- The ER model also has an associated diagrammatic representation, the ER diagram, which can express the overall logical structure of a database graphically.

7.2.1 Entity Sets (实体集, 实体型)

- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
 - Example:
$$\text{instructor} = (ID, name, street, city, salary)$$
$$\text{course} = (course_id, title, credits)$$
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.

Entity Sets (cont.)

- A entity consists of all values of its all attributes
 - e.g. Fig 7.1

- Entity-set

$$\begin{aligned} \text{customer-set} = \{ & \langle \text{ID-value}, \text{name-value}, \text{street - value}, \\ & \text{city -value} \rangle \\ & | \text{ID-value} \in D_1, \text{name-value} \in D_2, \\ & \text{street -value} \in D_3, \text{city -value} \in D_4 \} \\ & \subseteq D_1 \times D_2 \times D_3 \times D_4 \end{aligned}$$

Entity Sets

instructor_ID instructor_name

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

student-ID student_name

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

§ 7.2.2 Relationship Sets

- A **relationship** is an association among several entities
 - e.g. in Fig. 6.2

Jone
customer
entity set

borrower
relationship

L-17
loan
entity set

- A **Relationship set** (联系集, 联系型) is a set of relationship of the same type
 - *note*: more than one relationship set among the same entity sets

§ 7.2.2 Relationship Sets

- A **relationship** is an association among several entities

Example:

44553 (Peltier) advisor 22222 (Einstein)
student entity relationship set *instructor* entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

- Example:

$$(44553, 22222) \in \text{advisor}$$

Relationship Sets (cont.)

- The entity sets $E_1, E_2, \dots, E_n$ participate in relationship set R
- A relationship instance in an E-R schema represents an association between the named entities in the real-world enterprise that is being modeled
 - e.g. in Fig. 7.2, the relationship instance between *Crick* and the *Tanaka*

Relationship Set *advisor*

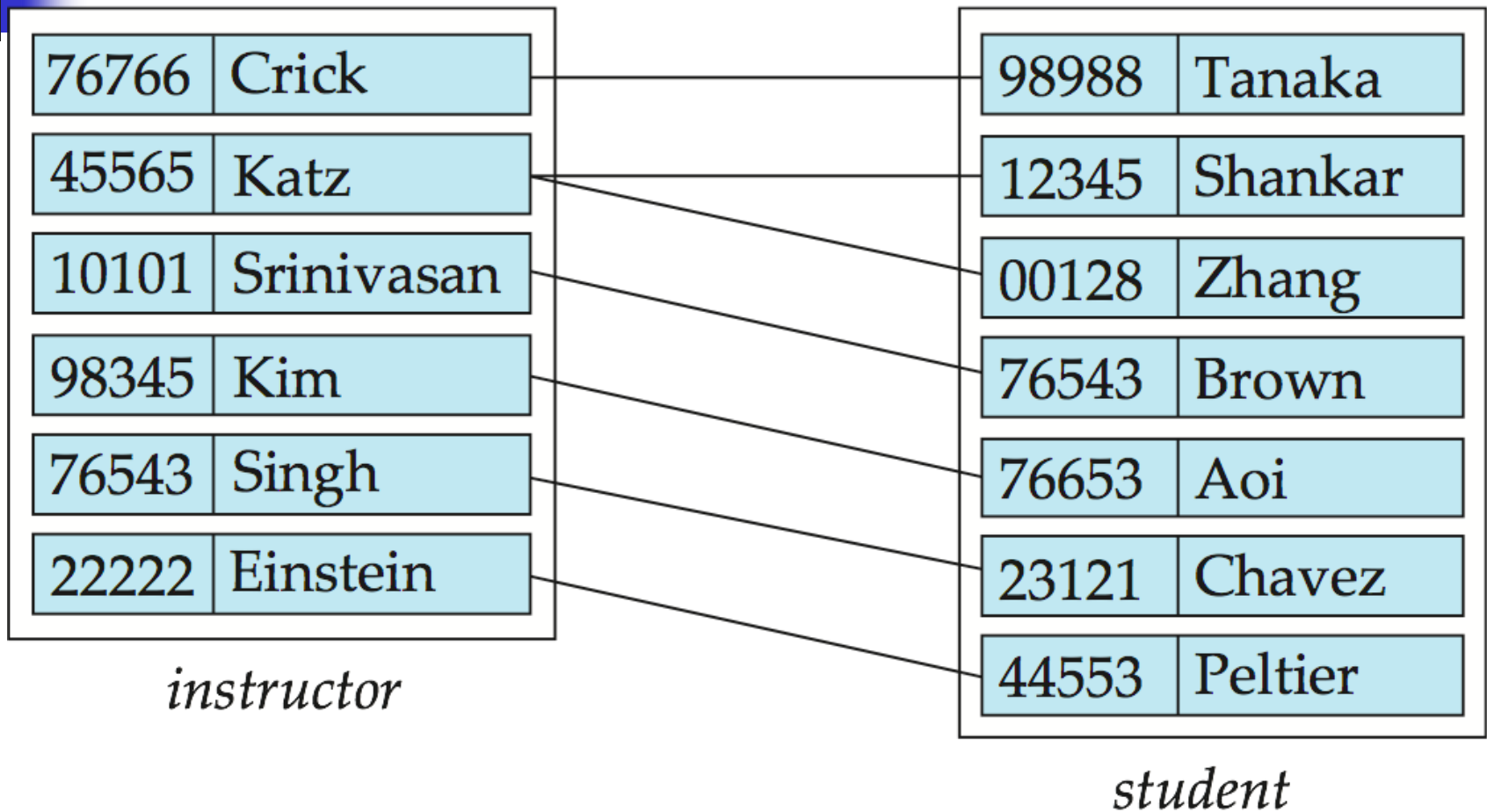
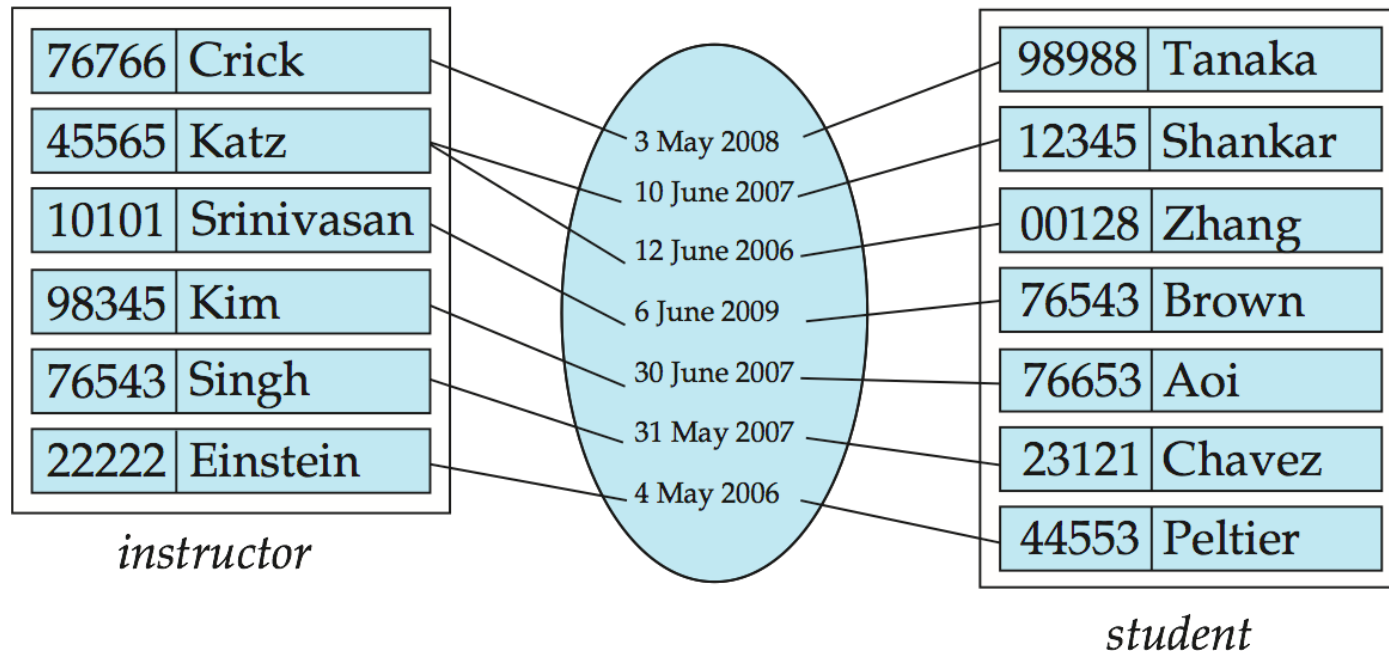


Fig.7.2

Relationship Sets (Cont.)

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor



Degree of a Relationship Set

- binary relationship
 - involve two entity sets (or degree two).
 - most relationship sets in a database system are binary.
- Relationships between more than two entity sets are rare. Most relationships are binary. (More on this later.)
 - ▶ Example: *students* work on research *projects* under the guidance of an *instructor*.
 - ▶ relationship *proj_guide* is a ternary relationship between *instructor*, *student*, and *project*

Relationship Sets (cont.)

- $\in \supseteq \subseteq$

- Role

- the functions that an entity plays in a relationship is called that entity's role



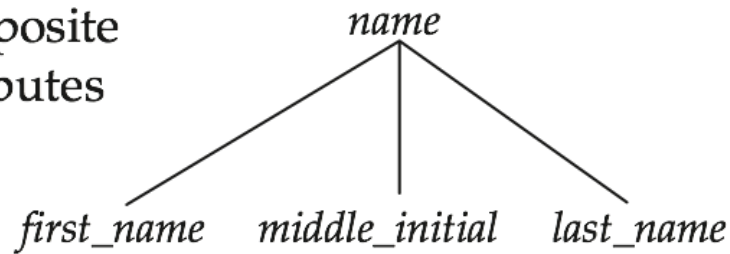
§ 7.2.3 Attributes

- The *domain* or *value set* of the attribute
 - the set of permitted values for the attribute

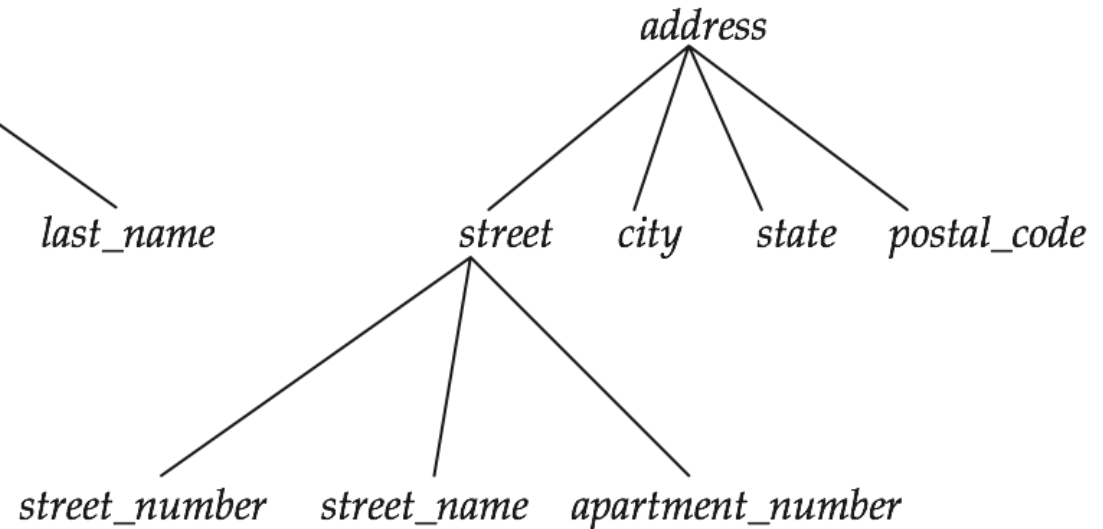
- Attribute types:
 - **Simple** and **composite** attributes.
 - **Single-valued** and **multivalued** attributes
 - Example: multivalued attribute: *phone_numbers*
 - **Derived** attributes
 - Can be computed from other attributes
 - Example: age, given date_of_birth

Composite Attributes

composite
attributes

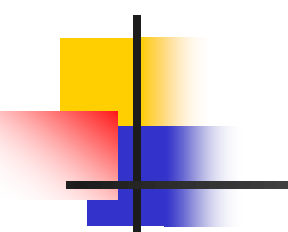


component
attributes



Attributes (cont.)

- Null value for a attribute means
 - the attribute “not applicable” for the entity, not existing
 - the value for the attribute exists, but is “unknown”



Redundant Attributes

- Suppose we have entity sets:
 - *instructor*, with attributes: *ID*, *name*, *dept_name*, *salary*
 - *department*, with attributes: *dept_name*, *building*, *budget*
- We model the fact that each instructor has an associated department using a relationship set *inst_dept*
- The attribute *dept_name* appears in both entity sets. Since it is the primary key for the entity set *department*, it replicates information present in the relationship and is therefore redundant in the entity set *instructor* and needs to be removed.
- BUT: when converting back to tables, in some cases the attribute gets reintroduced, as we will see later.

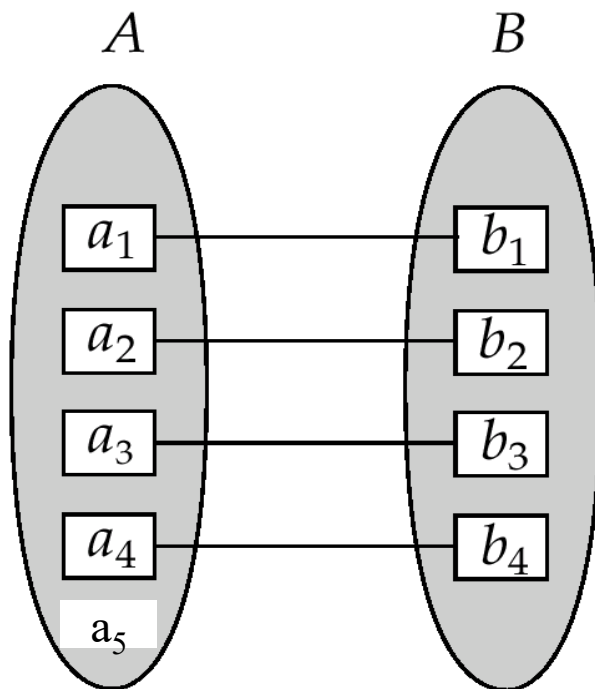
§ 7.3 Constraints

- Semantic/constraints
 - as *integrity constraints* in DBS, to maintain DBS *consistency*
- Integrity constraints in E-R model, on E or R
 - keys (§ 7.3.3)
 - mapping cardinalities (§ 7.3.1)
 - participation-constraints (§ 7.3.2)
 - cardinality limits

7.3.1 Mapping Cardinalities

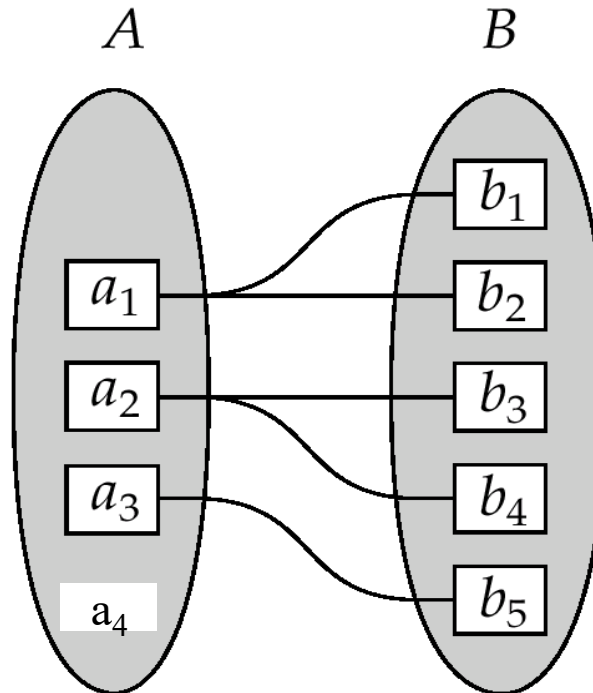
- Mapping cardinalities
 - *semi-quantitatively* expressing the number of entities to which another entity can be associated via a relationship set
- For a binary relationship set R , the mapping cardinality must be one of the following types: Fig.7.5, Fig.7.6, *from A to B*
 - one to one
 - *one to many*
 - *many to one*
 - many to many

—注意many-to-one和one-to-many 定义和方向！



(a)

One to one



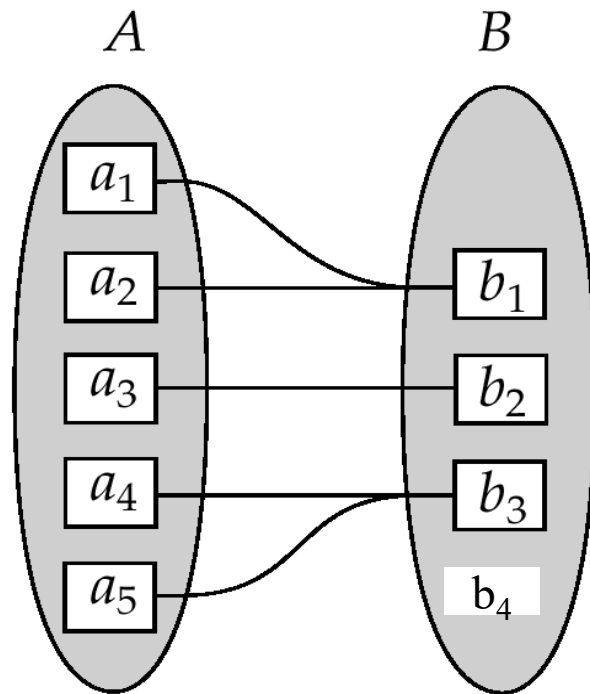
(b)

One to many

Note: some elements in A or B may not be mapped to any elements in the other set

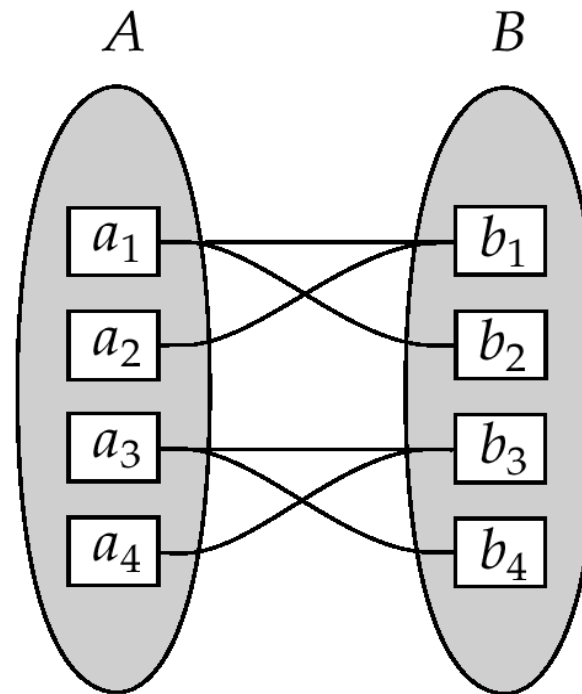
Fig. 7.5 Mapping cardinalities

—注意many-to-one和one-to-many 定义和方向！



(a)

Many to one



(b)

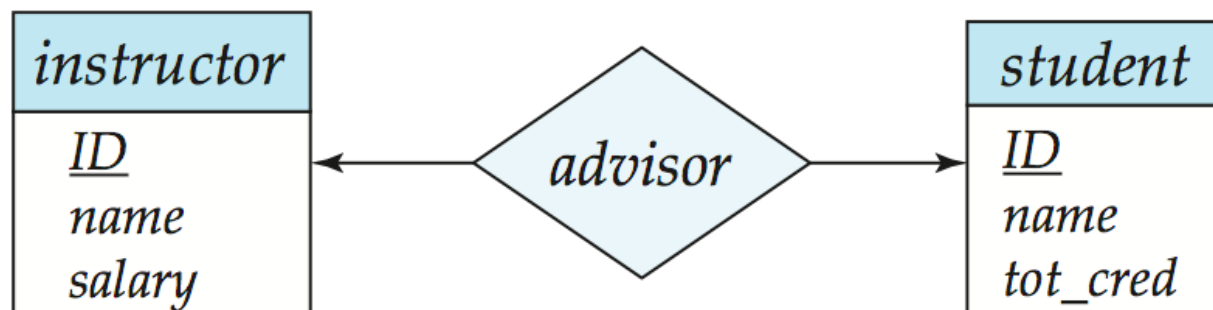
Many to many

Note: Some elements in A or B may not be mapped to any elements in the other set

Fig. 7.6 Mapping cardinalities

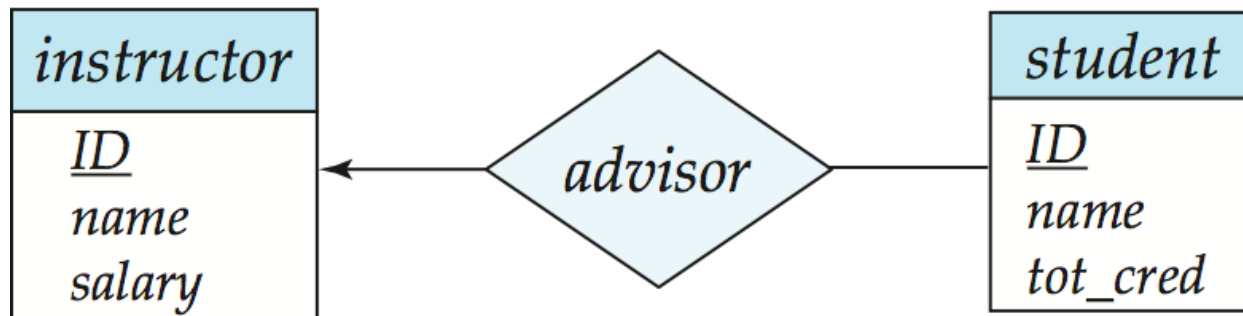
Cardinality Constraints

- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($—$), signifying “many,” between the relationship set and the entity set.
- One-to-one relationship between an *instructor* and a *student* :
 - A student is associated with at most one *instructor* via the relationship *advisor*
 - A *student* is associated with at most one *department* via *stud_dept*



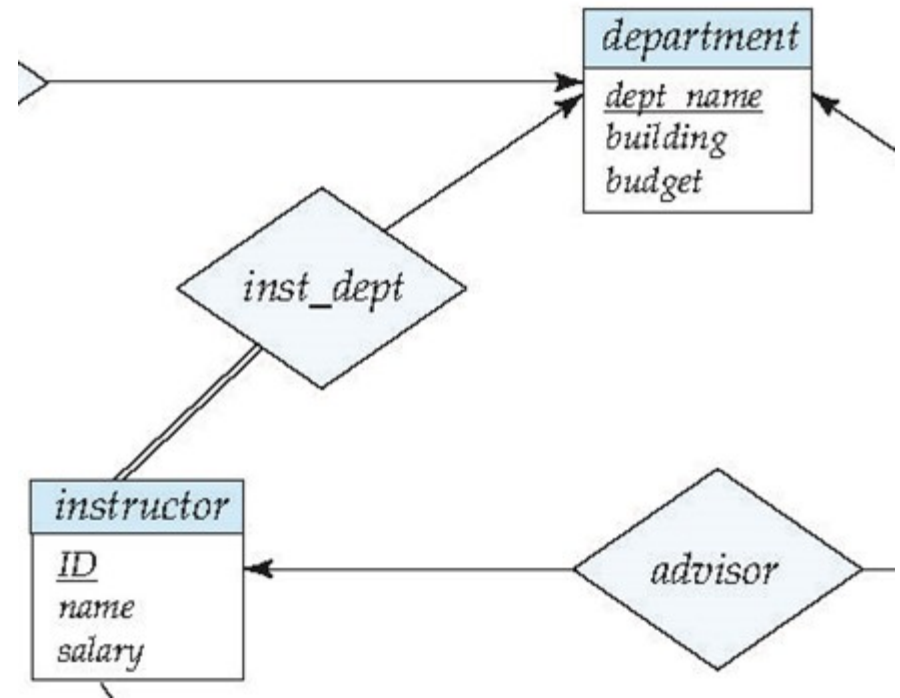
One-to-Many Relationship

- one-to-many relationship between an *instructor* and a *student*
 - an instructor is associated with several (including 0) students via *advisor*
 - a student is associated with at most one instructor via *advisor*,



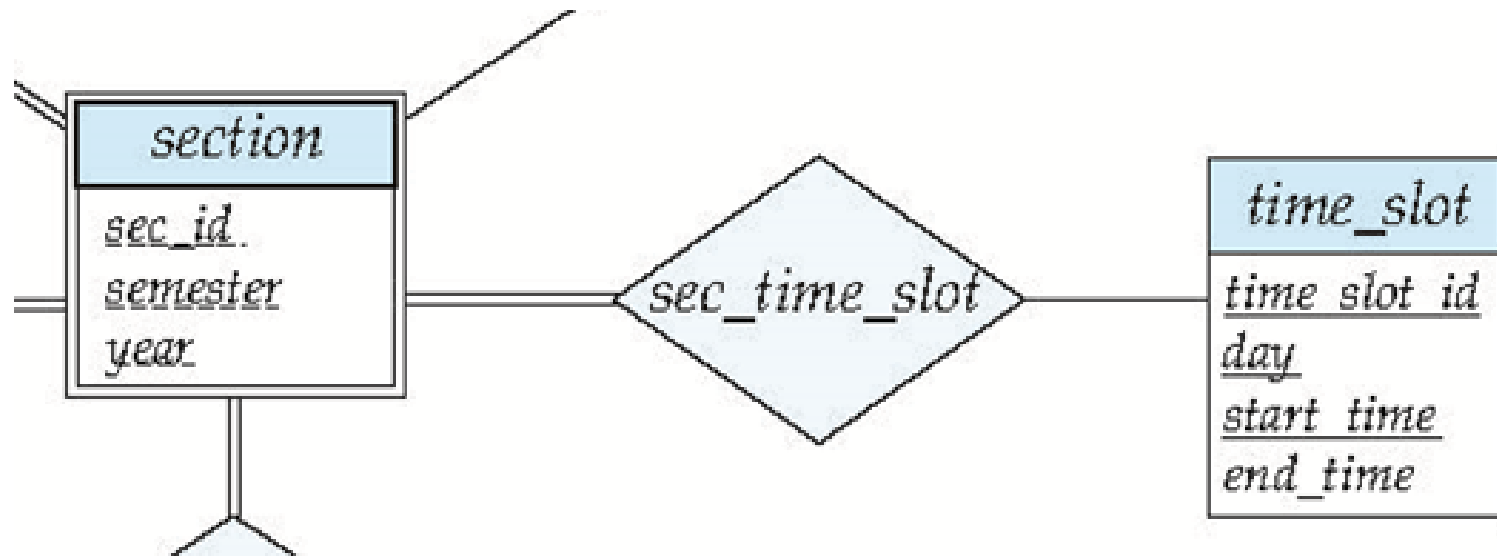
Many-to-One Relationships

- In a many-to-one relationship between an *instructor* and a *department*,
 - an instructor is associated with at most one department via *inst_dept*,
 - and a department is associated with several (at least one) instructor via *inst_dept*



Many-to-Many Relationship

- A *section* is associated with several (possibly 0) timeslots via *sec_time_slot*
- A *timeslot* is associated with several (possibly 0) sections via *sec_time_slot*



7.3.3 Keys

- **Key** is a set of attributes (of a *entity set* or *relationship set*)
 - , in which there are one or more attributes
 - the values of these attributes in one **entity** can be used to uniquely distinguish this entity from others , or
 - the values of these attributes in one **relationship** are used to uniquely identify the relationship
- Keys include
 - superkey (超键), candidate key (候选键), primary key (主键)

Keys For Entity Sets

- A *super key* of an entity set is a set of one or more attributes, whose values uniquely determine each entity in the entity set
 - e.g. $\{instructor_id, instructor_name\}$
 - the super key may contain extraneous attributes
 - e.g. *instructor_name*
- A candidate key is the minimal super key
 - *non-redundant* super key
 - e.g. *instructor-id* is the candidate key of *instructor*

Keys For Entity Sets (cont.)

- The *primary key* is a candidate key chosen by the database designer as the principal means of identifying entities within an entity set
 - although several candidate keys may exist, one of the candidate keys is selected to be the *primary key*
 - need to consider semantics of relationship set in selecting the *primary key* in case of more than one candidate key

构造关系表时，如果有多个候选键，最好选取数值型（**int**, **float**）候选键作为关系表主键，便于提高基于主键的查询速度

—不要选字符串型属性，如**varchar**、**datetime**

—e.g. **studentname**, **instructorName**

Keys For Relationship Sets

- Keys for relationship sets **R** on entities $E_1, E_2, \dots, E_n$
 - $R = \{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, \dots, e_n \in E_n\}$
 $\subseteq E_1 \times E_2 \times \dots \times E_n$
 , how to uniquely distinguish each relationship **instances** $\{(e_1, e_2, \dots, e_n) ?$
 - **R** is the combination of $E_1, E_2, \dots, E_n$, each E_i can be uniquely distinguished by $\text{primary_key}(E_i)$, $1 \leq i \leq n$, so the set of all attributes in $\text{primary_key}(E_1), \text{primary_key}(E_2), \dots, \text{primary_key}(E_n)$ can be used to recognize $(e_1, e_2, \dots, e_n)$

Keys For Relationship Sets (cont.)

- The **super**_key for R
 - $\text{primary_key}(E_1) \cup \text{primary_key}(E_2) \dots \cup \text{primary_key}(E_n)$
 - e.g. in Fig.7.3, $(\text{InstructorID}, \text{StudentIDr})$ is the super key of *advisor* ▶
 - *note*
 - if the attribute names of primary-keys are not unique, the **attributes with the same names** should be renamed
- The candidate keys for R
 - minimal, non-redundant super keys

Keys For Relationship Sets (cont.)

- The *candidate* keys or the *primary* key for a **binary** relationship set R among entity sets A and B can be decided as follows, in accordance with the mapping cardinality of R

- R is *many-to-many*, ►

$$\text{primary_key}(R) = \text{primary_key}(A) \cup \text{primary_key}(B)$$

- R is *many-to-one from A to B*, !!

$$\text{primary_key}(R) = \text{primary_key}(A)$$

- R is *one-to-many*, !! ►

$$\text{primary_key}(R) = \text{primary_key}(B)$$

Keys For Relationship Sets (cont.)

- R is *one-to-one*, 

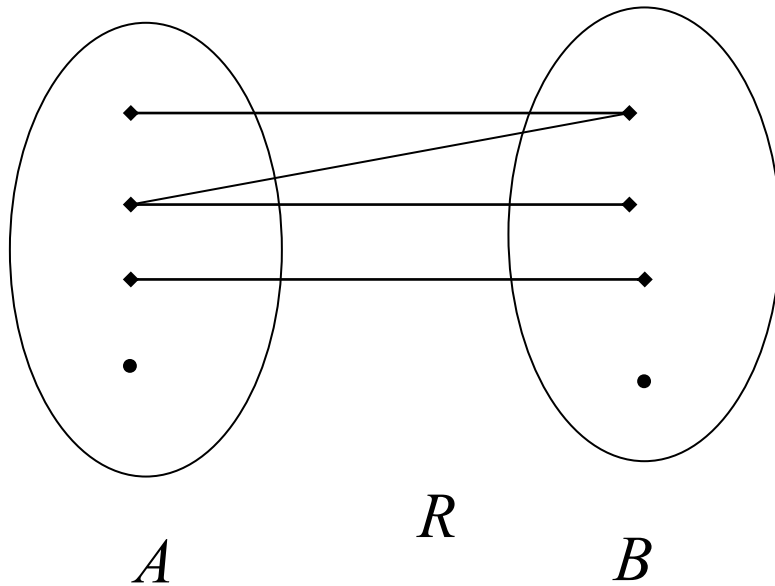
$$\text{primary_key}(R) = \text{primary_key}(A)$$

$$\text{or: } \text{primary_key}(R) = \text{primary_key}(B)$$

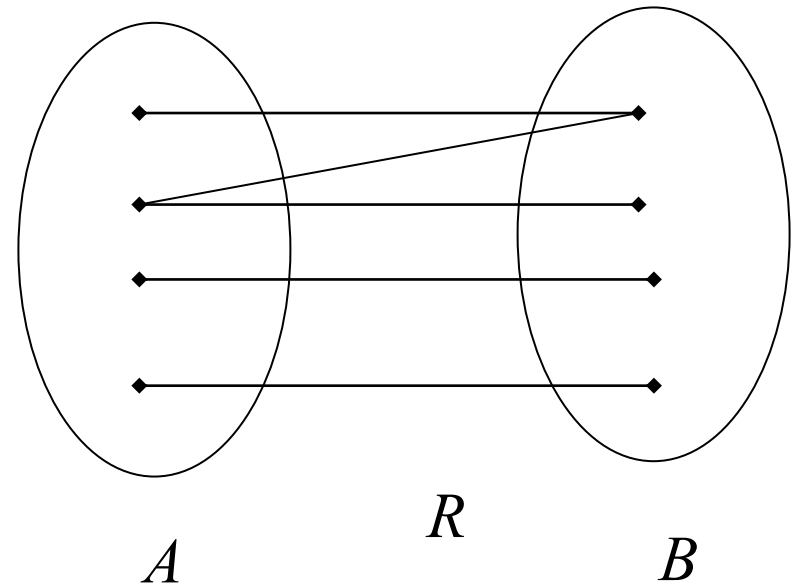
7.3.3 Participate Constraints

- The participation of an entity E in a relationship R is *total*, if
 - every entity in E participates in at least one relationship in R
 - e.g. participation of *instructor* in *inst_dept* is *total*
 - refer to Fig.7.15 
 - every *instructor* **must** have a *department* associated to it via *inst_dept*
- The participation of an entity E in a relationship R is *partial*, if
 - some entities in E may not participate in any relationship in R
 - e.g. participation of *student* in *advisor* is *partial*, because some students maybe have no advisors
 - refer to Fig.7.15 

Participate Constraints (cont.)



(a) partial participation



(b) total participation

Fig. 7.0.4 Total/participation participation

Cardinality Limits for Participation

(§ 7.5.2, pp277)

- Cardinality limits (参与的基数界限) are used to express *quantitative* constraints on participation
- E.g. *instructor*, *student*, *advisor*
 - 每个 *student* 最少有 1 个指导 *instructor*, 最多也只有 1 个指导 *instructor*
 - 每个 *instructor* 最多可以指导多个 *student*, 最少可以指导 0 个 *student*

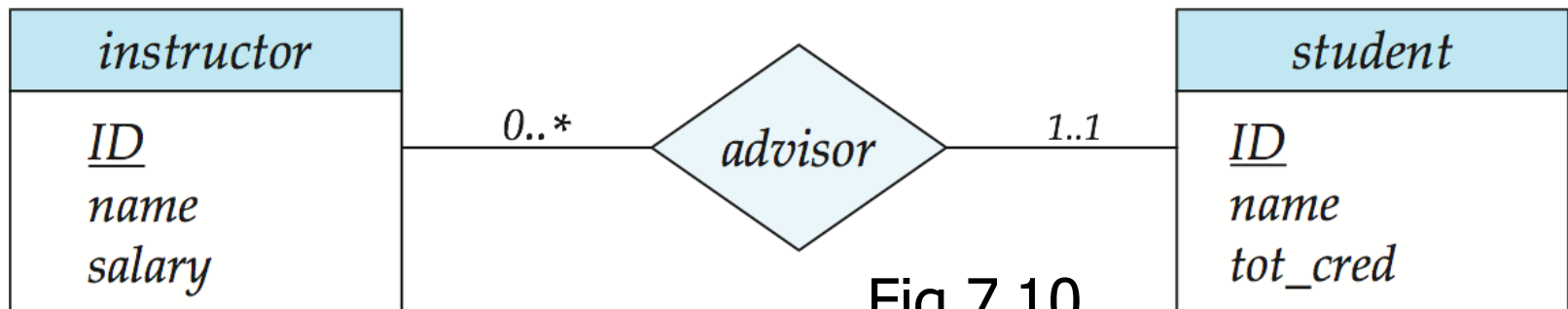


Fig.7.10

Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors

Cardinality Limits for Participation

- /* 设联系 R 关联了 entity sets A 和 B , 为定量地描述 A 参与 R 的 total/partial participation 和 A 中的 entity 与 B 中的 entity 的 mapping cardinality, 引入实体参与联系的 cardinality limits
- A 参与 R 的基数下界 l_A 和上界 h_A , refer to Fig.7.0.5
 - A 中的每个实体 a 通过 R 关联了最少 l_A 个、最多 h_A 个 B 中实体 b
 - l_A : 对 A 中的每个实体 a , B 中至少有 l_A 个实体 b 通过 R 与其对应/关联
 - h_A : 对 A 中的每个实体 a , B 中至多有 h_A 个实体 b 通过 R 与其对应/关联

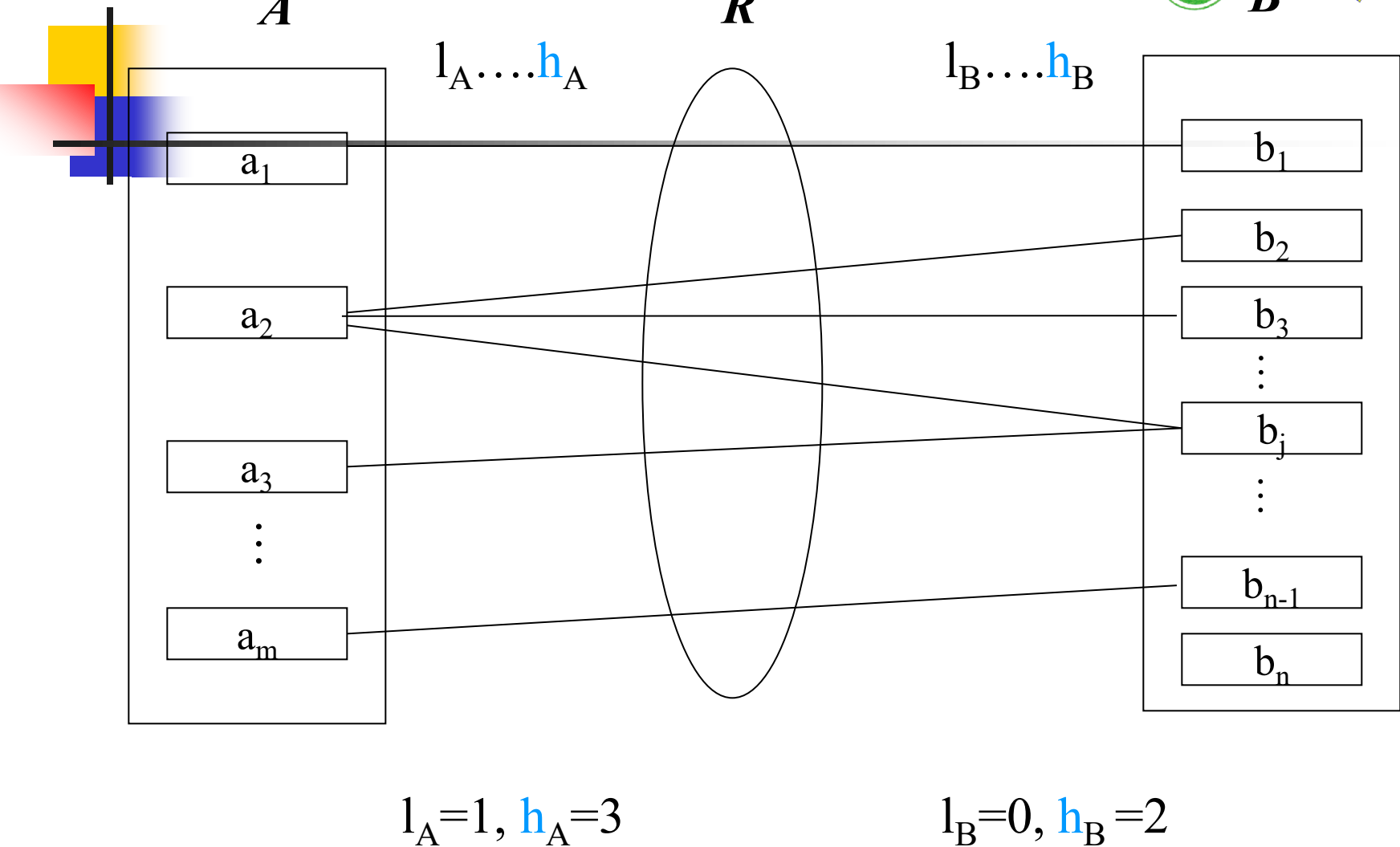


Fig.7.0.5 Illustration for cardinality limits

Cardinality Limits for Participation

(cont.)

- Note
 - A maximum value of * indicates no limit
 - *in some textbooks*, $\langle l_A, h_A \rangle$ is put at the side of the entity **B**
- Cardinality limits vs total/partial participation
 - A minimum value $l_A = 0$: *A* is partial participation of **R**
 - A minimum value $l_A > 0$: *A* is total participation of **R**, equivalent to *double line*

Cardinality Limits for Participation

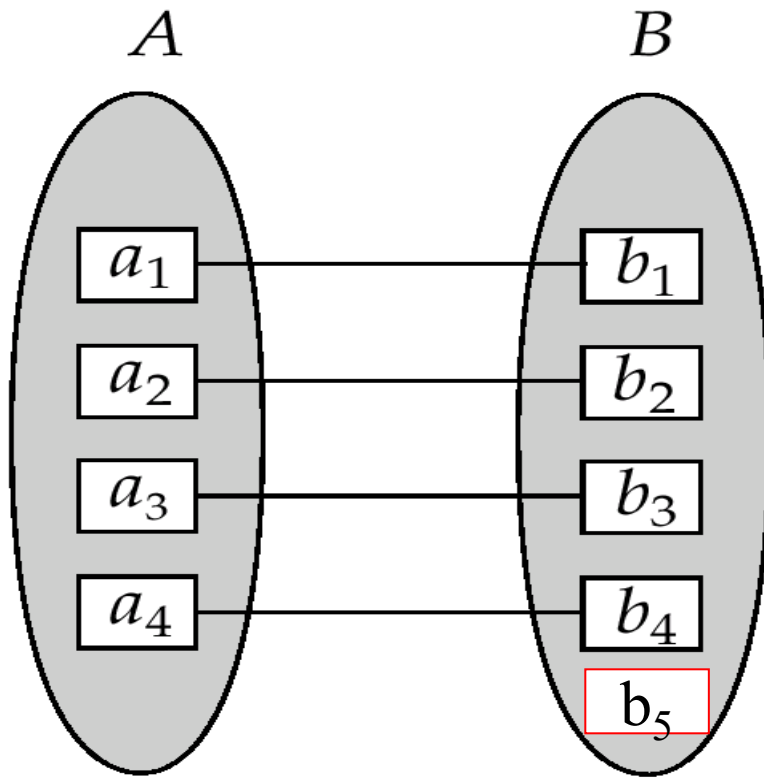
(cont.)

- Cardinality limits vs mapping cardinality
 - 设联系 R 关联了实体集 A 和 B , 利用 A 的基数界限 $\langle l_A, h_A \rangle$ 中的 h_A 、 B 的基数界限 $\langle l_B, h_B \rangle$ 中的 h_B , 可推导出联系 R 的映射基约束
 - $\langle h_B, h_A \rangle$ 表示了联系 R 的从 A 到 B 的映射基约束 !!!!
 - e.g. in Fig.7.10 , considering mapping cardinality form *instructor* to *student*
 - mapping cardinality form *instructor* to *student* depends on $\langle h_{\text{student}}, h_{\text{instructor}} \rangle$
 - for student, $h_{\text{student}}=1$, for instructor, $h_{\text{instructor}}=*$
 - so, $\langle h_{\text{student}}, h_{\text{instructor}} \rangle = \langle 1, * \rangle$, and is *one to many*

Cardinality Limits for Participation

(cont.)

- The associations between cardinality limits and the mapping cardinality are classified as follows, and illustrated by Fig.7.0.6

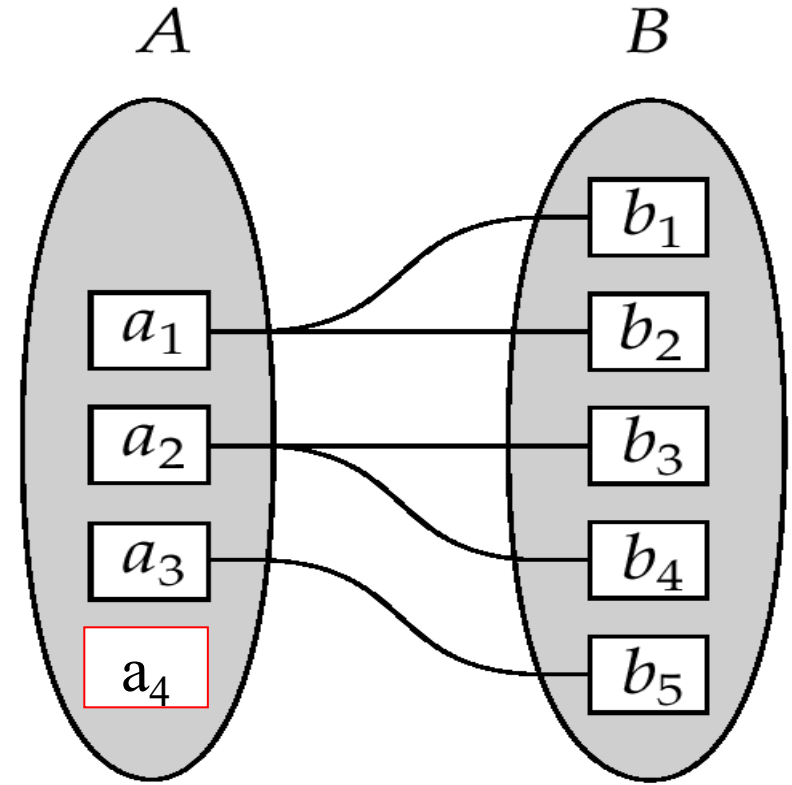


(a)

From A to B , **one** to **one** (1:1),

$$\langle l_A, h_A \rangle = \langle 1, \mathbf{1} \rangle,$$

$$\langle l_B, h_B \rangle = \langle 0, \mathbf{1} \rangle;$$



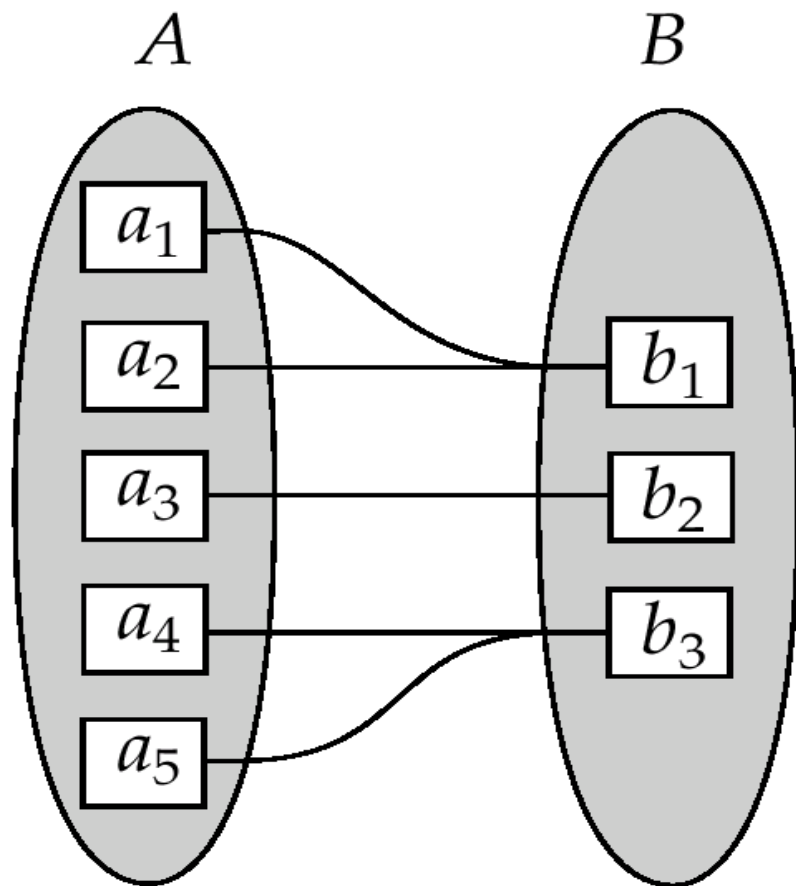
(b)

From A to B , **one** to **many** (1:2),

$$\langle l_A, h_A \rangle = \langle 0, \mathbf{2} \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, \mathbf{1} \rangle$$

Fig.7.0.6-1 Associations between cardinality limits and mapping cardinality

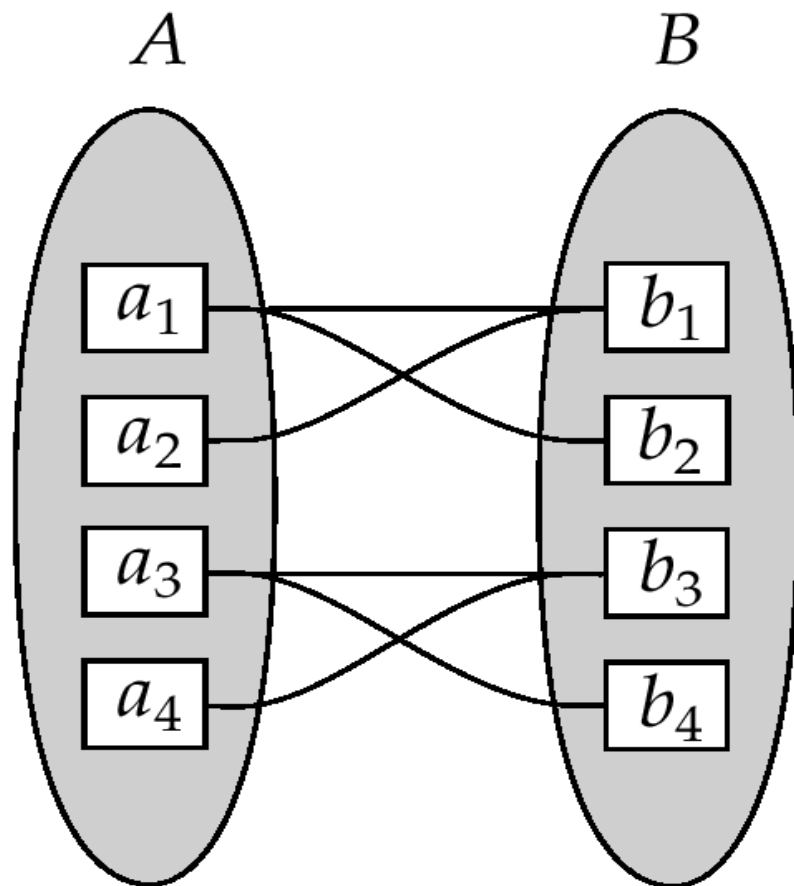


(a)

Form A to B , many to one (2:1),

$$\langle l_A, h_A \rangle = \langle 1, 1 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$



(b)

Form A to B , many to many (2:2)

$$\langle l_A, h_A \rangle = \langle 1, 2 \rangle$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$

§ 7.5.6 Weak Entity Sets

- An entity set that does not have a primary key is referred to as a *weak entity set*
 - e.g. *section(sec_id, semester, year)* in Fig.7.14 in next slide, and Fig. 7.0.10 
- A weak entity set E_1 can **only** be distinguished through another (**strong**) entity set E_2 , called **identifying (标识) /owner (属主) entity set**, which has association/relationship R with E_1
 - E_2 owns E_1 , E_1 must be related to E_2 via an instance of R , called the **identifying relationship (标识性联系)**.
 - R is **many-to-one** (or **one-to-one**) from the weak entity set E_1 to the identity set E_2

Weak Entity Sets (cont.)

- E_1 is of *total participation in* the relationship set R
- R is depicted using a double diamond
- weak entity set is depicted as double rectangles.

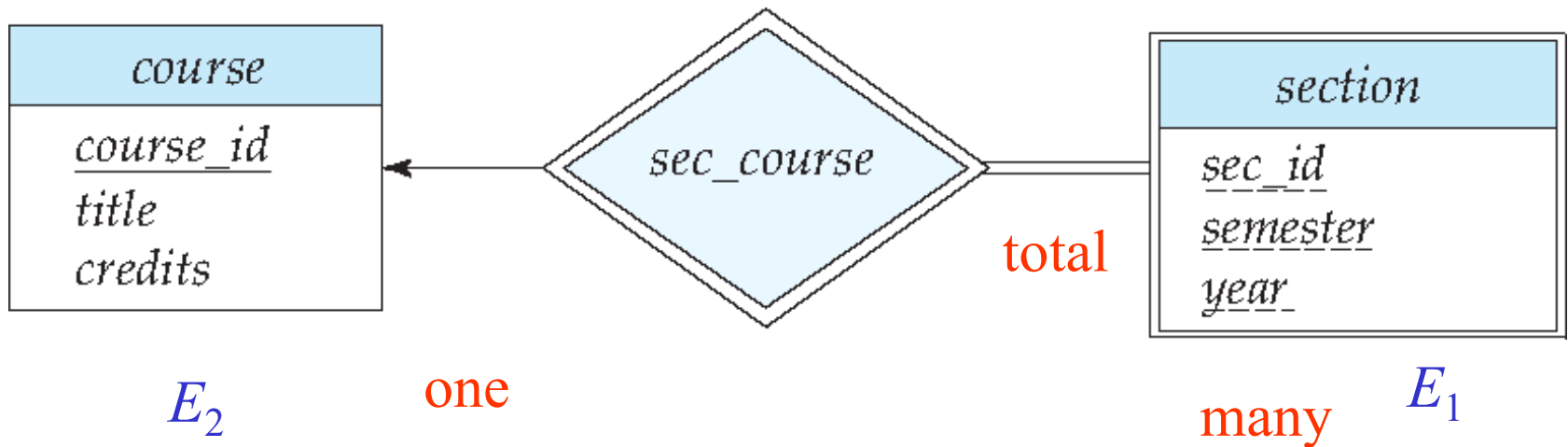


Fig.7.14 E-R diagram with a weak entity set

E_2 :course

E_1 :section/course-offering

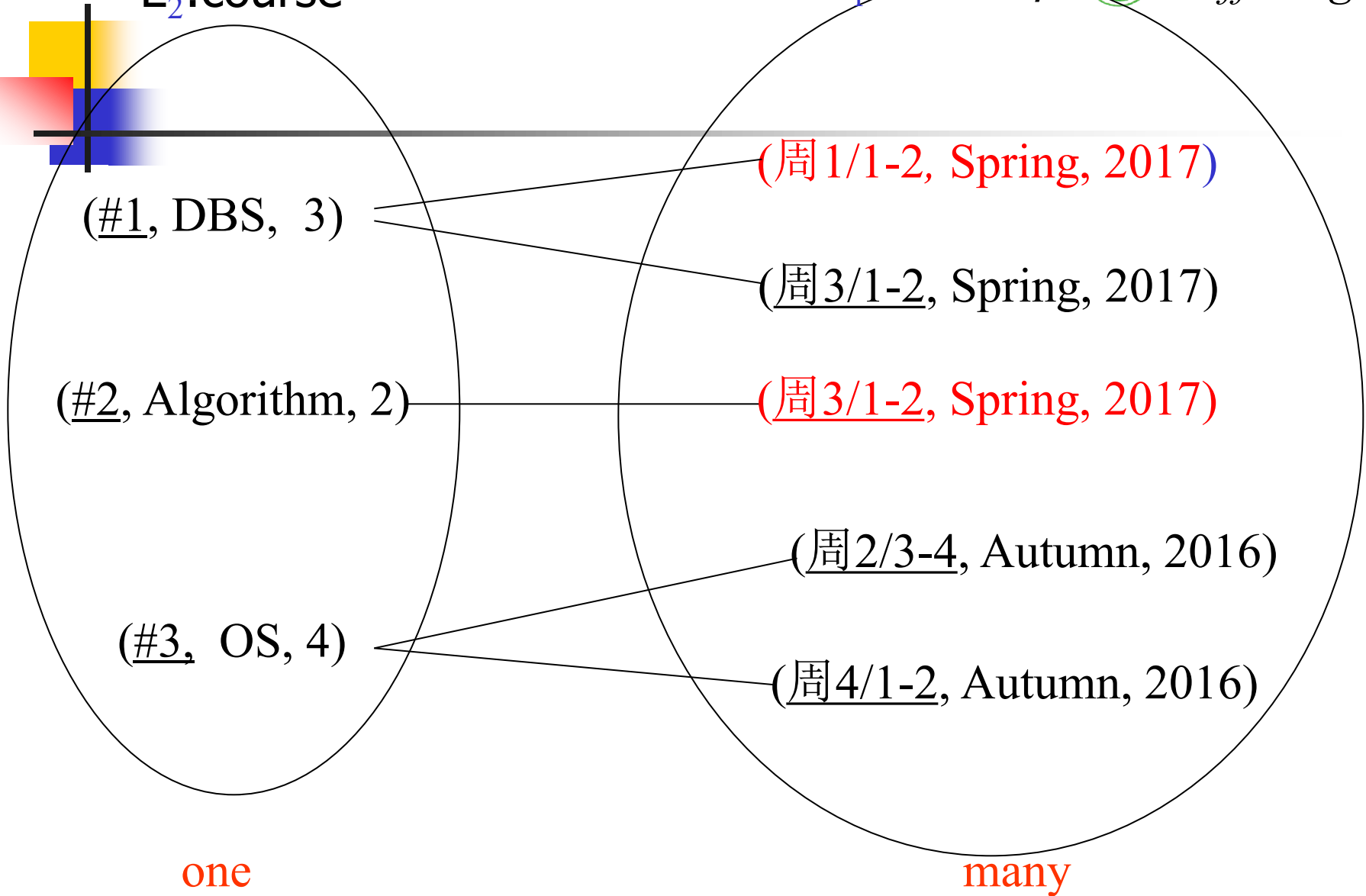


Fig.7.0.10 A example of weak entity set

Weak Entity Sets (cont.)

- The *discriminator* (分辨符) (or *partial key*) of a weak entity set is the set of attributes that distinguishes among all those entities in the weak entity set that depending on a particular strong entity
 - e.g. *sec_id*, *semester*, *year*
 - the discriminator of a weak entity set is underlined with a *dashed line* (虚线)

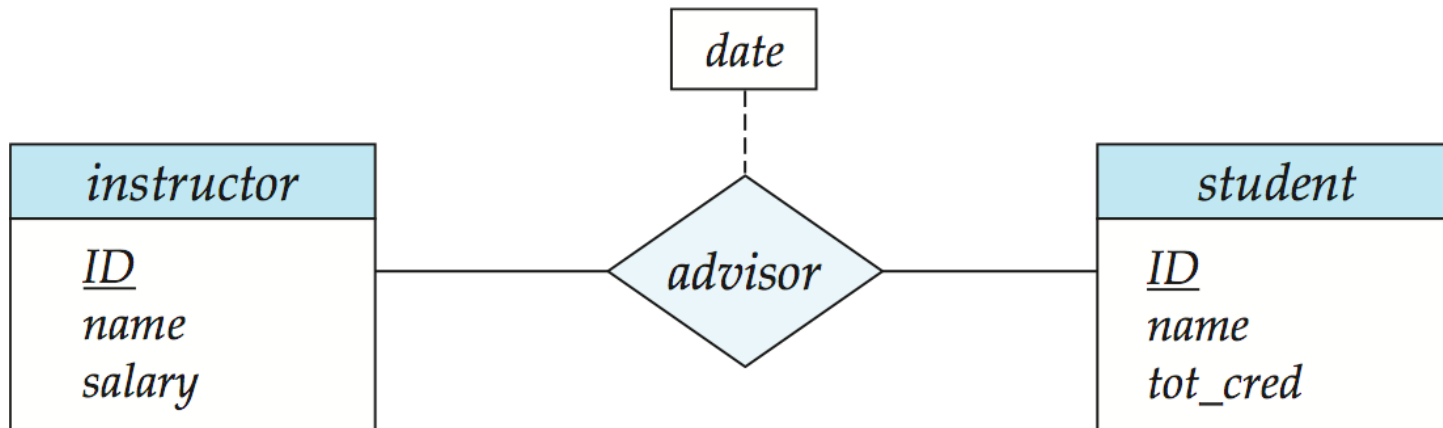
Weak Entity Sets (cont.)

- The *primary key of a weak entity set* is formed by the primary key of the *strong entity set* on which the weak entity set is existence dependent, plus the weak entity set's discriminator
 - primary key for weak entity set E_1
= discriminator $\cup$ primary_key(E_2) .
 - e.g. sec_id, semester, year, course_id
- Weak entity set E_1 with several identifying entity sets $E_2, E_3, \dots, E_n$, primary key for weak entity set E_1 =
discriminator $\cup$ primary_key(E_2) $\cup \dots$ primary_key(E_n) .

§ 7.5 E-R diagrams

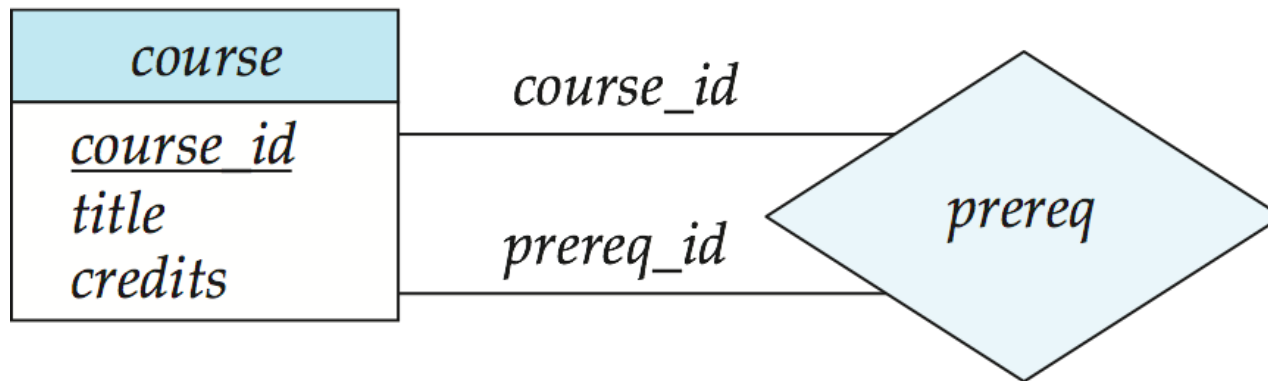
- *Rectangles* (矩形) represent entity sets
- *Diamonds* (菱形) represent relationship sets
- *Lines* link attributes to entity sets and entity sets to relationship sets.
- *Ellipses* (椭圆) represent attributes
 - *double ellipses* represent multivalued attributes
 - *Dashed* (虚线) *ellipses* denote derived attributes (派生属性)
- *Underline* indicates primary key attributes
- Double rectangles represent weak entity sets

Relationship Sets with Attributes



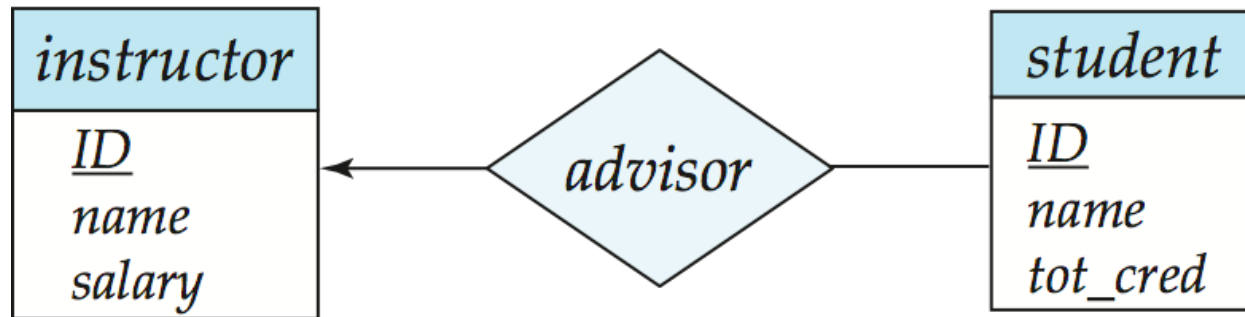
Roles

- Entity sets of a relationship need not be distinct
 - Each occurrence of an entity set plays a “role” in the relationship
- The labels “*course_id*” and “*prereq_id*” are called **roles**.



Cardinality Constraints

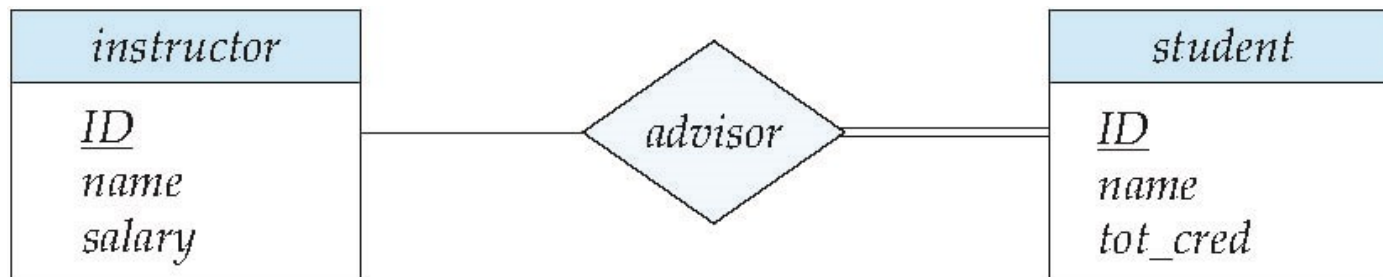
- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($—$), signifying “many,” between the relationship set and the entity set.



One-to-Many Relationship

Total and Partial Participation

Total participation (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor* relation is total

- ▶ every *student* must have an associated instructor

Partial participation: some entities may not participate in any relationship in the relationship set

Example: participation of *instructor* in *advisor* is partial

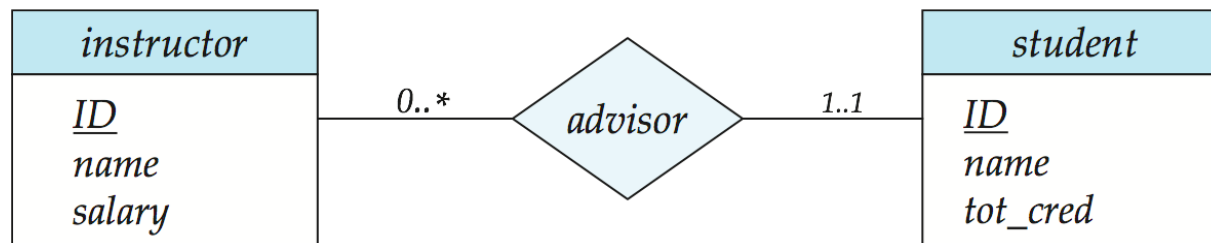
Cardinality Limits

A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality

A minimum value of 1 indicates total participation.

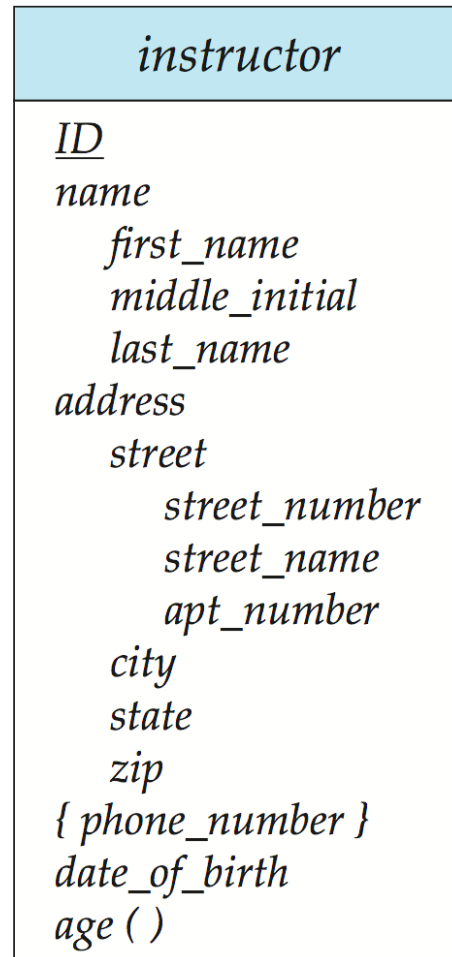
A maximum value of 1 indicates that the entity participates in at most one relationship

A maximum value of $*$ indicates no limit.



Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors

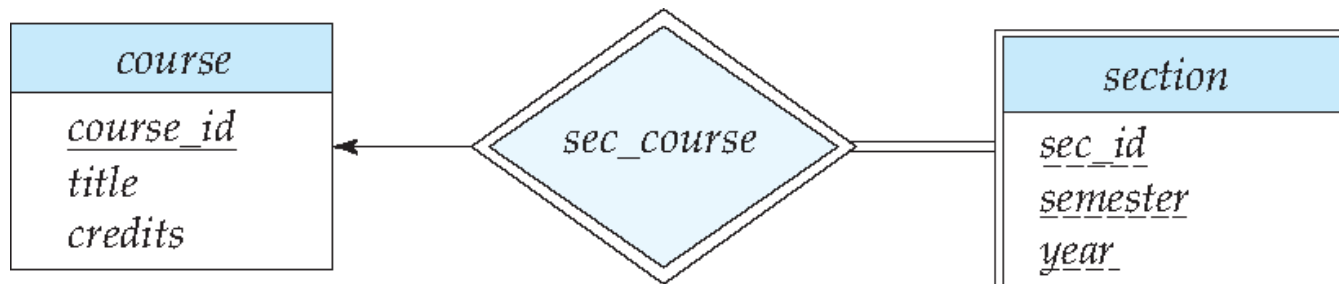
Entity with Complex Attributes



Expressing Weak Entity Sets

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* :

(*course_id*, *sec_id*, *semester*, *year*)



E-R diagrams (cont.)

■ 标注规则!!!

- 由联系发出的有向箭头指向one 所对应的实体，无向线段指向many端所对应的实体。
- 只有在many-to-many的情况下，E-R图中联系与实体间的线段 全部是无向线段，其他三种情况下必出现有向线段

§ 7.7 E-R Design Issues

7.7.1 Use of entity sets vs. attributes

- Mapping “*thing/object*” in real world into *entity* sets or *attribute* sets

- e.g. for *instructor* and *phone* objects in real world

- model 1. entity:

instructor(*ID*, *name*, *salary*, *phone-number*)

implying that employees **have precisely one** phone number each, unless *phone-number* is multivalued;

how to represent the other properties of “phone” object ?

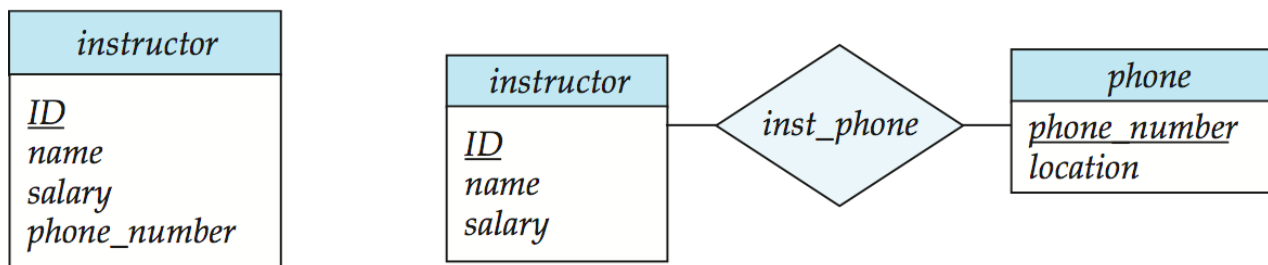


Fig.7.17 Alternatives for adding phone to the instructor entity set

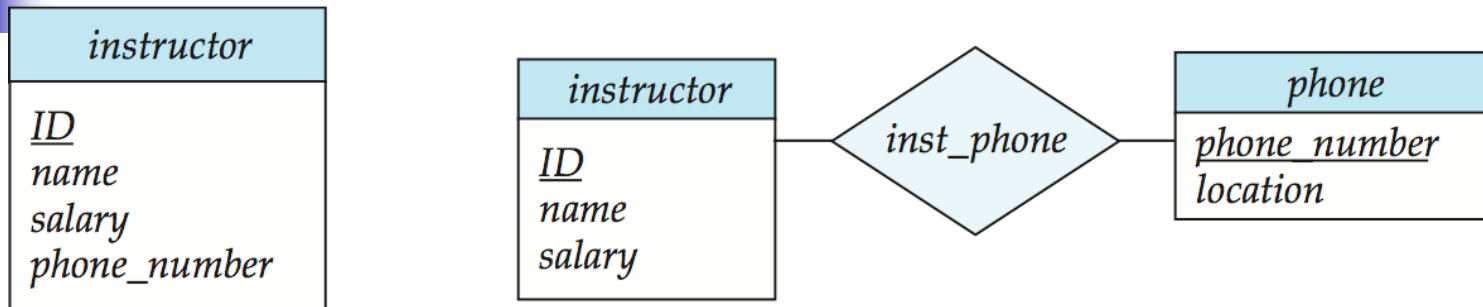


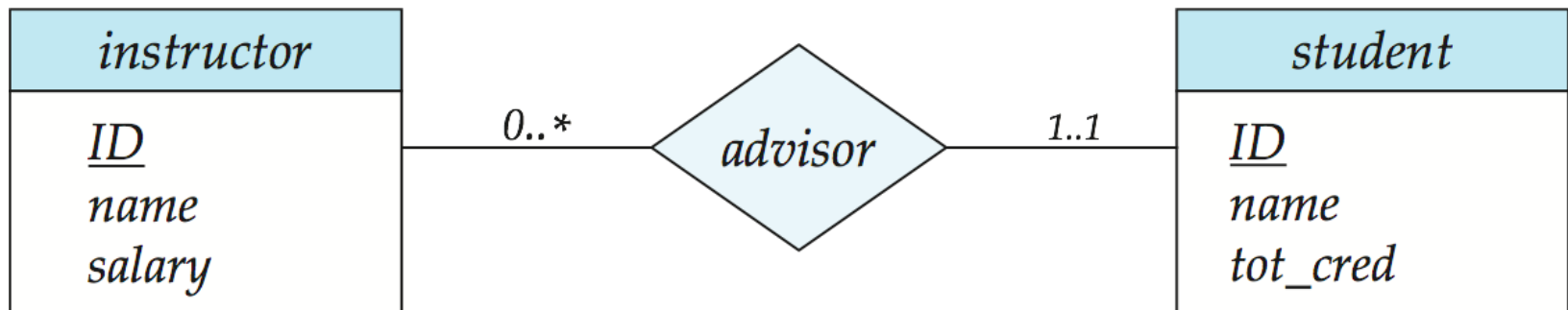
Fig.7.17 Alternatives for adding phone to the instructor entity set

- model 2. entity: *instructor*(*ID*, *name*, *salary*)
phone(*phone-number*, *location*, ...)
relationship: *inst-phone*

- Treating *phone* as an entity is more general, more information about *phone* can be modeled, e.g.
 - *Location*
 - plus multiple phone numbers

Entity Sets vs. Attributes (cont.)

- **Rule1.** Not to use the primary-key of an entity set as an attributes of another entity set (to represent *implicitly association* between these two entity sets), it is better to use an relationship set to *explicitly* show this *association*
 - e.g. for the entity *student*, do not use *instructor.id* as its attribute, instead of, representing association among *student* and *instructor* using relationship *advisor*



Entity Sets vs. Attributes (cont.)

- **Rule2.** If E participate in R , not to designate the primary-key(E) as attributes of R , to avoid *information redundancy*
 - for example, an *ill* modeling of *advisor*

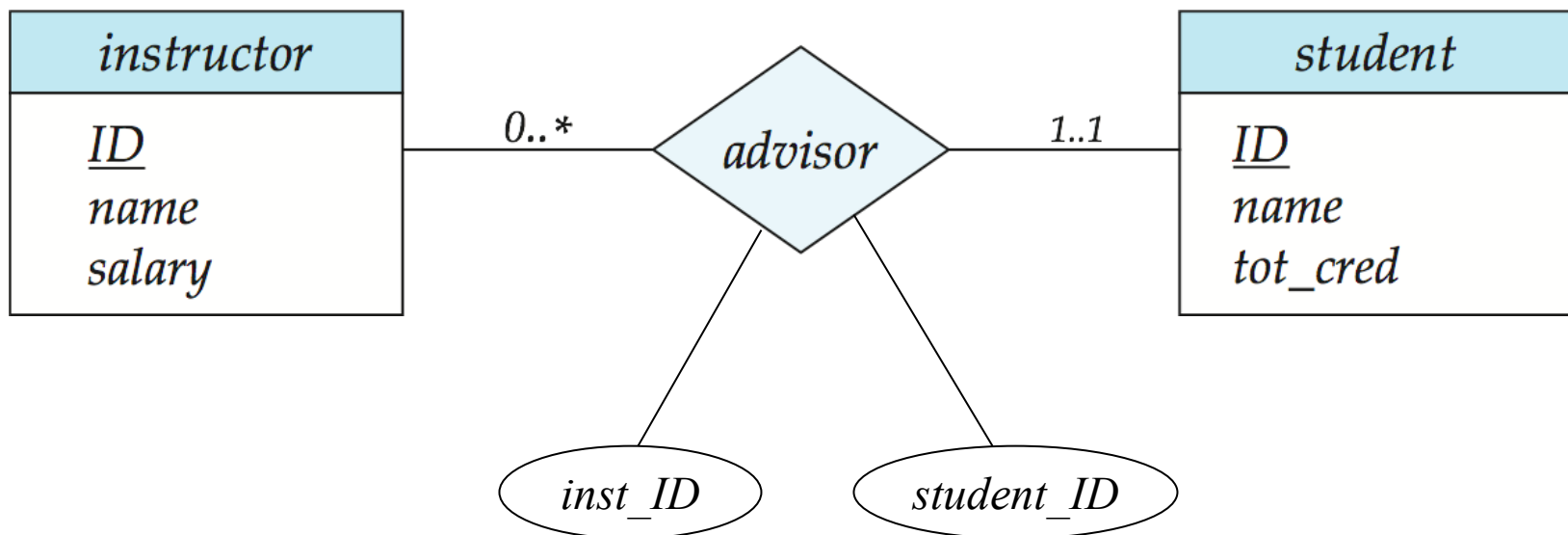


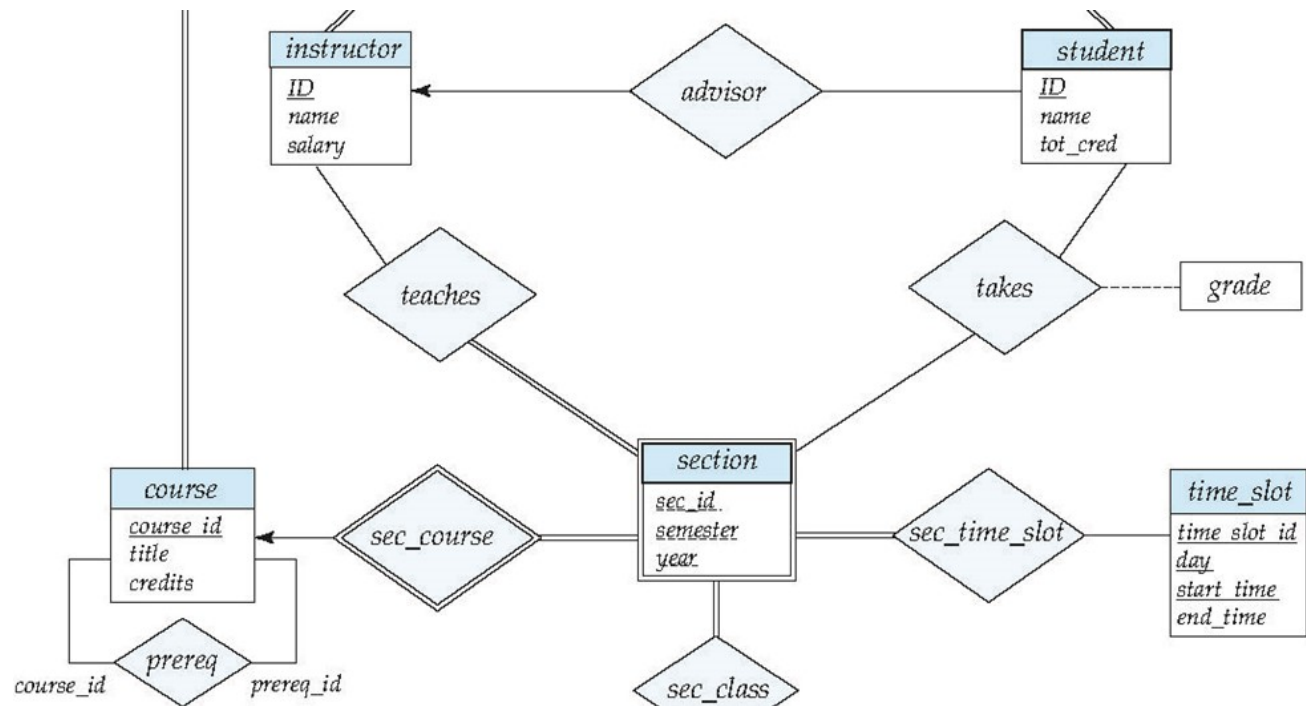
Fig.7.0.7 Anti-example:

Data redundancy in *advisor*

7.7.2 Use of Entity Set vs. Relationship Set

- **Rule3.** Use the relationship set to describe the *action* occurring between entities
 - e.g. *students* *select* *courses*

- In Fig.7.15 the *takes* relationship is used to model the situation where a *student* takes a *section* of a *course*



Entities vs. Relationship sets

- An alternative is to design a course-registration record, represented by a entity set named *registration*, for each course that each student takes

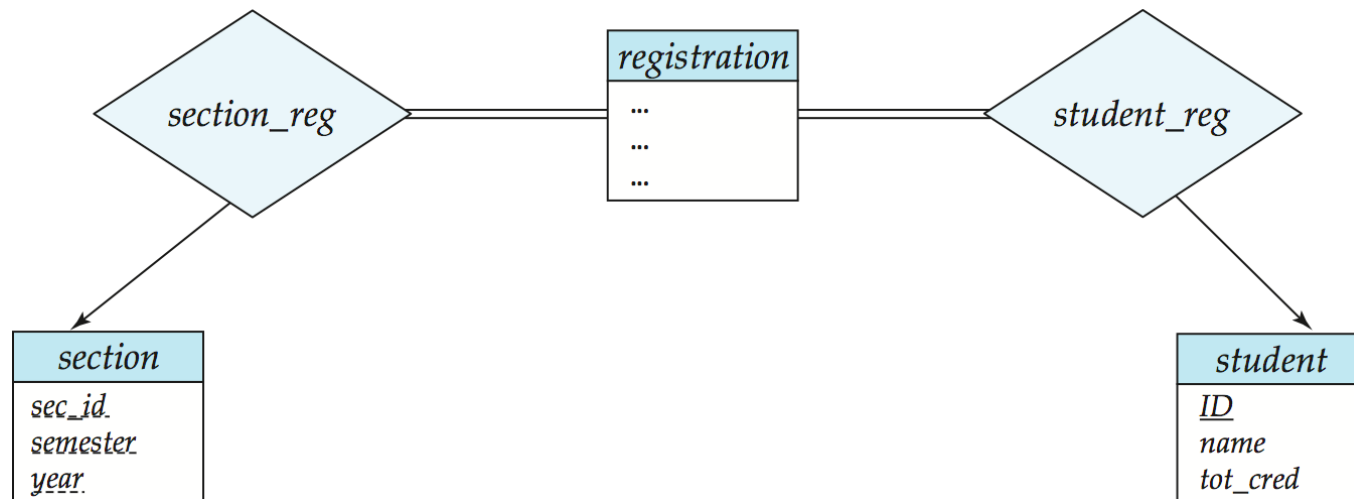
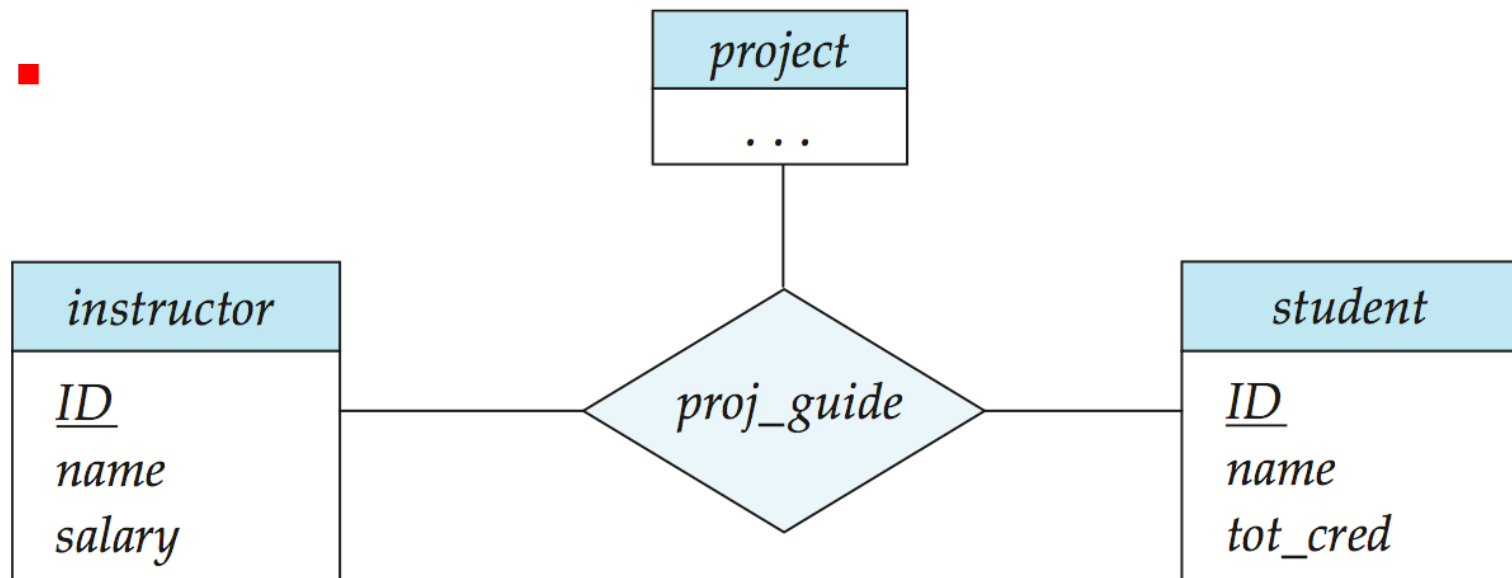


Fig.7.18

- In Fig.7.18, more information can be associated with the entity set *registration* than the relationship set *takes*

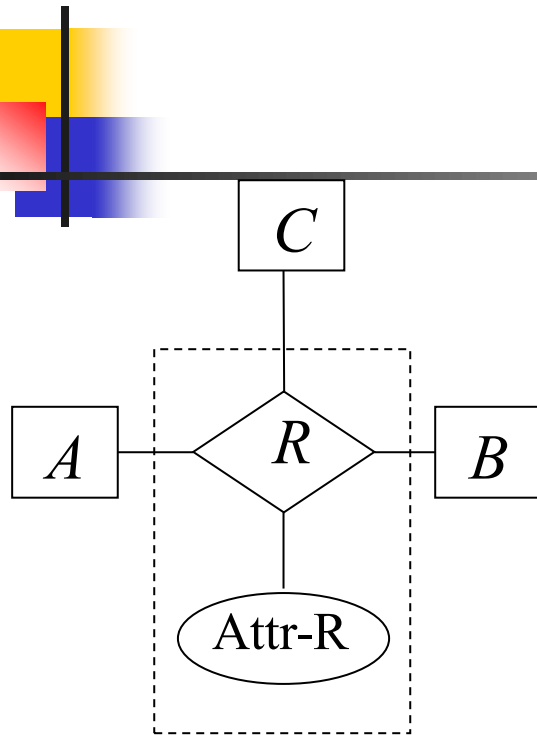
7.7.3 Binary vs N-ary Relationship Sets

- **Rule4.** *Prefer to* using the binary relationship set, and replace a non-binary relationship set by a number of binary relationship set if possible
 - *child-father-mother* is replaced with *child-father*, *child-mother* and *husband-wife*

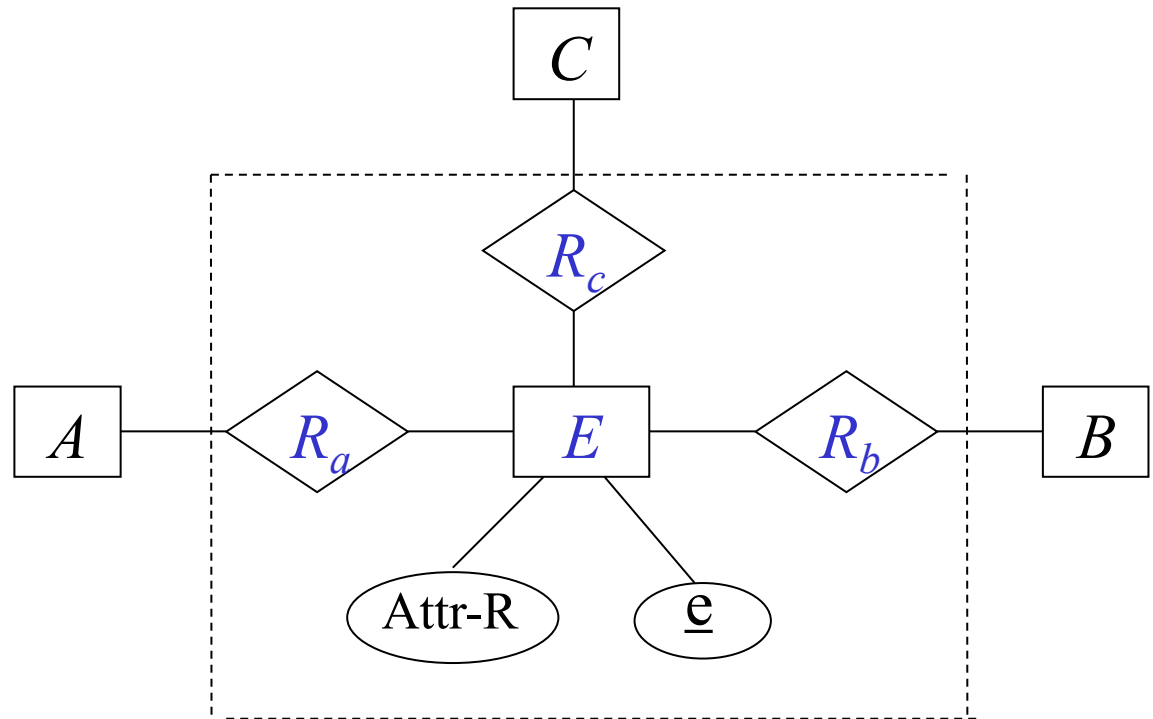


Binary vs N-ary Relationship Sets

- Translating a non-binary relationship set into several binary relationship sets
 - for 3-ary relationship set $R = \{(a_i, b_i, c_i) \mid a_i \in A, b_i \in B, c_i \in C\}$ among entity sets A, B, C , i.e. $R \subseteq A \times B \times C$, replace R by an entity set $E = \{e_i\}$ and three binary relationship sets R_a, R_b, R_c



(a)



(b)

Fig.7.0.8 Non-binary relationship set translating

Binary vs N-ary Relationship Sets (cont.)

- $E = \{ e_i \}$, $|E| = |R|$, i.e. **each** (a_i, b_i, c_i) in R corresponds to **one** e_i in E , or $E = \{ e_i \} = R = \{ (a_i, b_i, c_i) \}$
- $R_a = \{ (e_i, a_i) \mid e_i \in E, a_i \in A \}$, relating E and A
- $R_b = \{ (e_i, b_i) \mid e_i \in E, b_i \in B \}$, relating E and B
- $R_c = \{ (e_i, c_i) \mid e_i \in E, c_i \in C \}$, relating E and C
- E has an identifying attribute $\underline{e}$ (candidate key) to distinguish each e_i in E
- all attributes of R , i.e. attr- R , are assigned to E
- refer to Fig.7.0.8 (a) , (b)



Cardinality Constraints on Ternary Relationship

略

We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint

For example, an arrow from *proj_guide* to *instructor* indicates each student has at most one guide for a project

If there is more than one arrow, there are two ways of defining the meaning.

For example, a ternary relationship R between A , B and C with arrows to B and C could mean

1. Each A entity is associated with a unique entity from B and C or
2. Each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B

Each alternative has been used in different formalisms

To avoid confusion we outlaw more than one arrow

7.7.4 Placement of Relationship Attributes

- The attributes of a relationship sets R may be associated with one of its participating entity sets
- For the attribute *attr-A* in relationship set R between E_1 and E_2 ($R \subseteq E_1 \times E_2$, if the cardinality of R is
 - *one-to-one* : *attr-A* can be assigned as the attribute of either E_1 or E_2 , rather than relationship set R
 - *one-to-many* from E_1 to E_2 : *attr-A* can only be assigned as the attribute of the entity set E_2 (the many-side), rather than the relationship set R

Placement of Relationship Attributes

- e.g. the relationship *advisor* in Fig.7.20 from *instructor* to *student* is *one to many*, the attribute *date* of *advisor*, which specifies when the instructor became the advisor of the student, can be made as an attribute of *student*, instead of the attribute of the relationship *advisor*

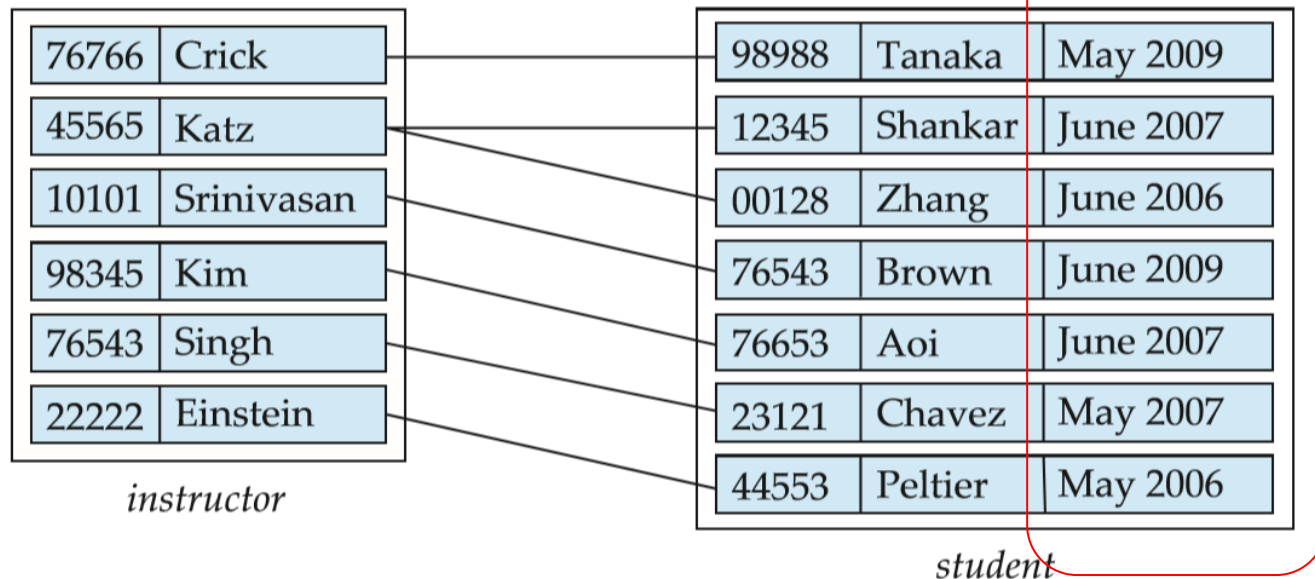


Figure 7.20 *date* as an attribute of the *student* entity set.

Placement of Relationship Attributes

- *many-to-many*: **attr-A** may only be assigned as the attribute of R , not of its participating entity sets E_1 or E_2

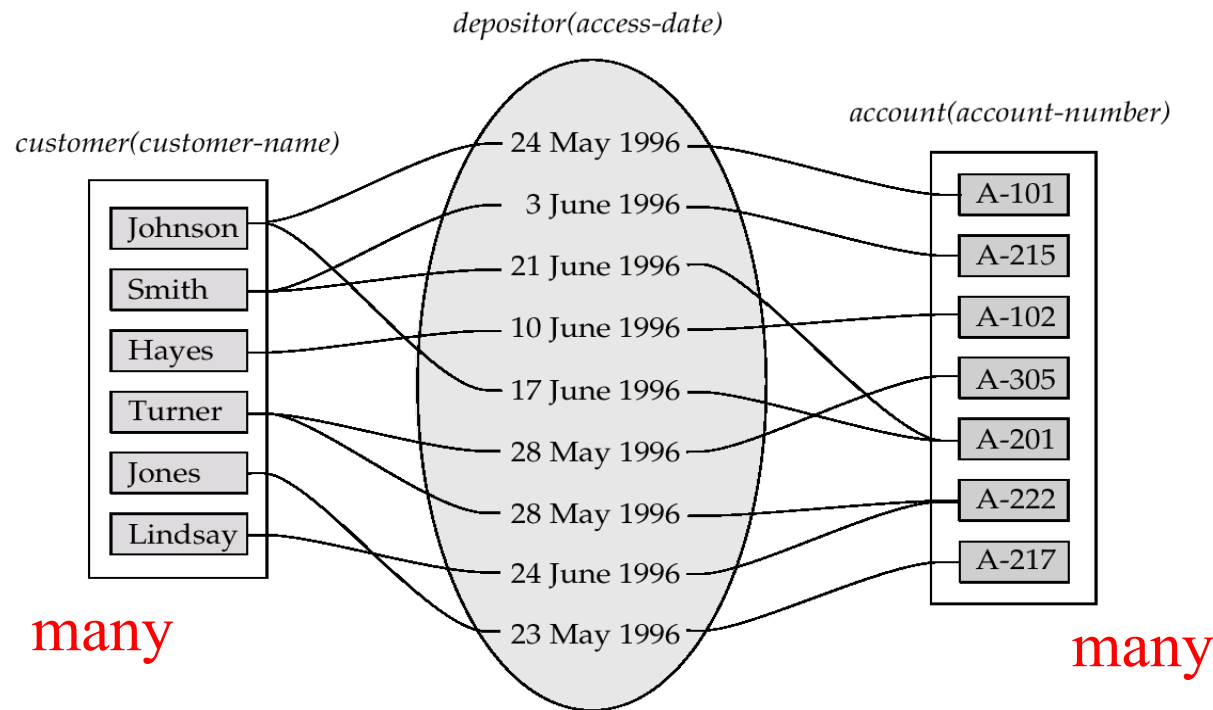


Fig.8.0.9 *Access-date* as attribute of the *depositor* relationship set

§ 7.8 Extended E-R model

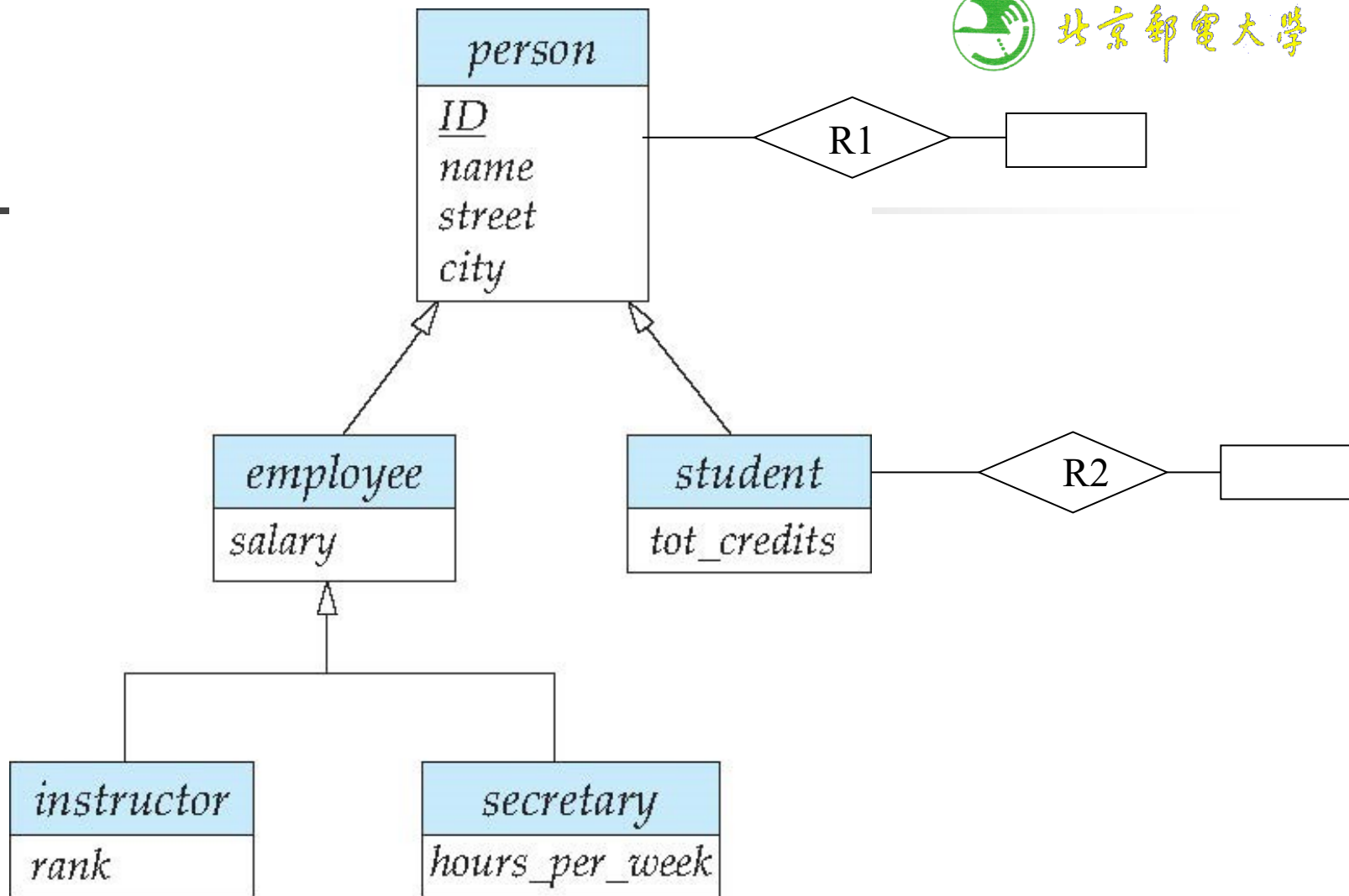
- Object-oriented (OO) E-R
 - specialization (特化, 特殊化, 例化)
 - generalization (概括化, 泛化, 普遍化)
 - attributes inheritance (属性继承)

7.8.1 Specialization

- Specialization
 - **top-down** design process of designating subgroups within an entity set, according to **differences** between entities in the entity set, each subgroup is distinctive from other entities in the set

Specialization (cont.)

- Refer to Fig.7.21
- These subgroupings become lower-level entity sets that **may** have attributes or **may** participate in relationships that do not apply to the higher-level entity set
- In the E-R model, **specialization is depicted by a triangle (三角形) component labeled *ISA***
 - e.g. *student* “is a” *person* in Figure 7.21



Overlapping – *employee* and *student*

Disjoint – *instructor* and *secretary*

Fig. 7.21 Specialization and generalization

7.8.2 Generalization

■ Generalization

- a bottom-up design process that combines a number of entity sets that **share the same features** into a higher-level entity set
- e.g. Fig.7.21
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way
 - the terms *specialization* and *generalization* are used interchangeably

Superclass vs Subclass

- The *ISA* relationship, or *containment*, is also referred to as **superclass-subclass** relationship
- **Superclass and subclass** 
 - for the entity sets A and B , if A is the generalization of B , i.e. B is the specialization of A , then A is the superclass of B , and B is the subclass of A

7.8.3 Attribute Inheritance

- A lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked
 - a higher-level entity's attributes and relationship can apply to all of its lower-level entities ▶

7.8.4 Constraints on Generation/Specialization

- How to determine the membership of subclass/lower-level entities, i.e., how to divide superclass/higher-level entity set into lower-level entity sets
 - *condition-defined*, attribute-defined generalization
 - on the basis of explicit condition/predicate/attributes-value
e.g. *student_type*= graduate, undergraduate
 - *user-defined*
 - e.g. all instructors are divided into four teams based on their assigned works, and each team becomes a subclass of the *instructor* entity set

Constraints on Specialization/Generalization

- **Completeness constraint** -- specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization. ►
 - **total**: an entity must belong to one of the lower-level entity sets, e.g. *student* generalization
 - **partial**: an entity need not belong to one of the lower-level entity sets, e.g. *person* specialization
- Partial generalization is the default. We can specify total generalization in an ER diagram by adding the keyword **total** in the diagram and drawing a dashed line from the keyword to the corresponding hollow arrow-head to which it applies (for a total generalization), or to the set of hollow arrow-heads to which it applies (for an overlapping generalization).
- The *student* generalization is total: All student entities must be either graduate or undergraduate. Because the higher-level entity set arrived at through generalization is generally composed of only those entities in the lower-level entity sets, the completeness constraint for a generalized higher-level entity set is usually total

Constraints on Generation/Specialization (cont.)

- depicted in E-R diagram as

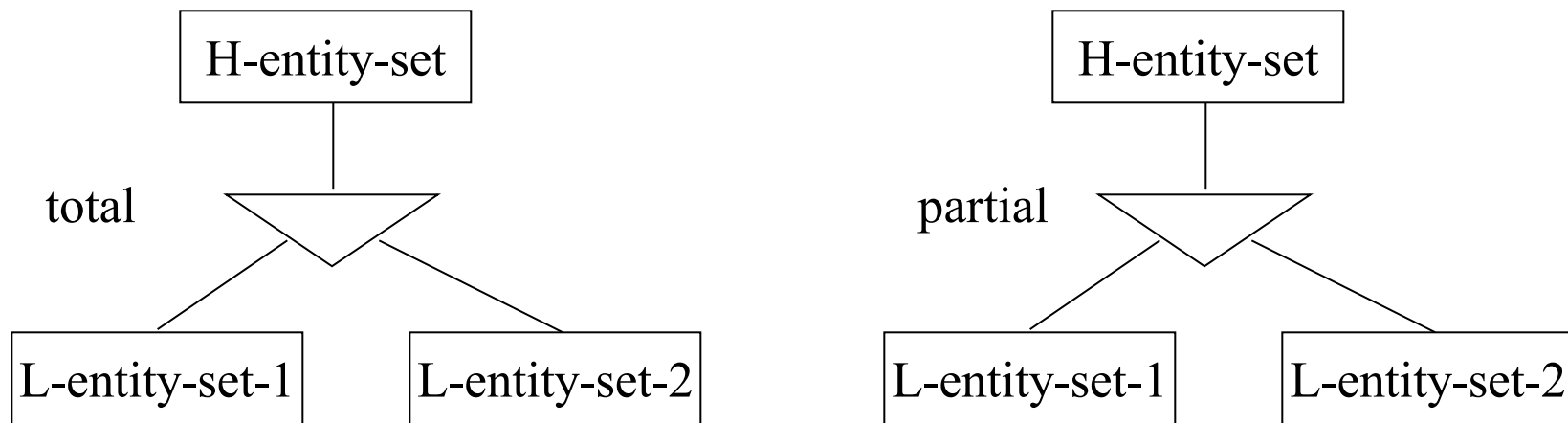


Fig.7.0.12 Completeness constraints

6.7.4 Constraints on Generation/Specialization (cont.)

- Constraints on whether or not a high-level entity may belong to more than one lower-level entity set *within a single specialization*, e.g. in Fig. 7.21
 - disjoint 
 - a high-level entity can belong to only one lower-level entity set, i.e., $L\text{-entity-set-1} \cap L\text{-entity-set-2} = \Phi$
 - E.g. *instructor* and *secretary*

Constraints on Generation/Specialization (cont.)

- overlapping
 - a high-level entity can belong to more than one lower-level entity sets
 - $L\text{-entity-set-1} \cap L\text{-entity-set-2} \neq \Phi$
 - *E.g. employee and student*
- depicted in E-R diagram as

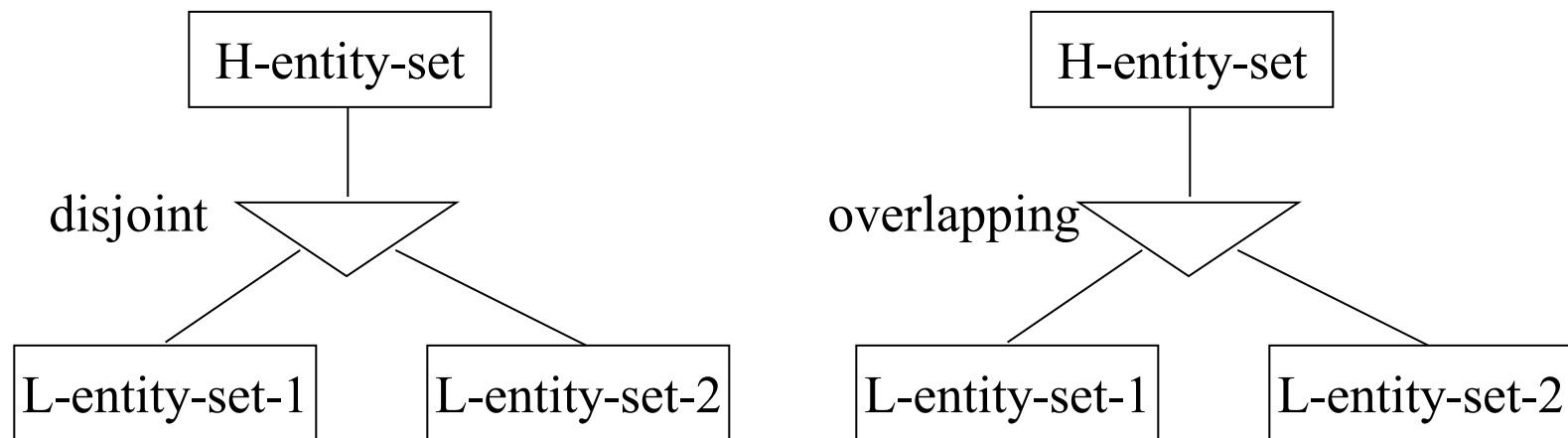
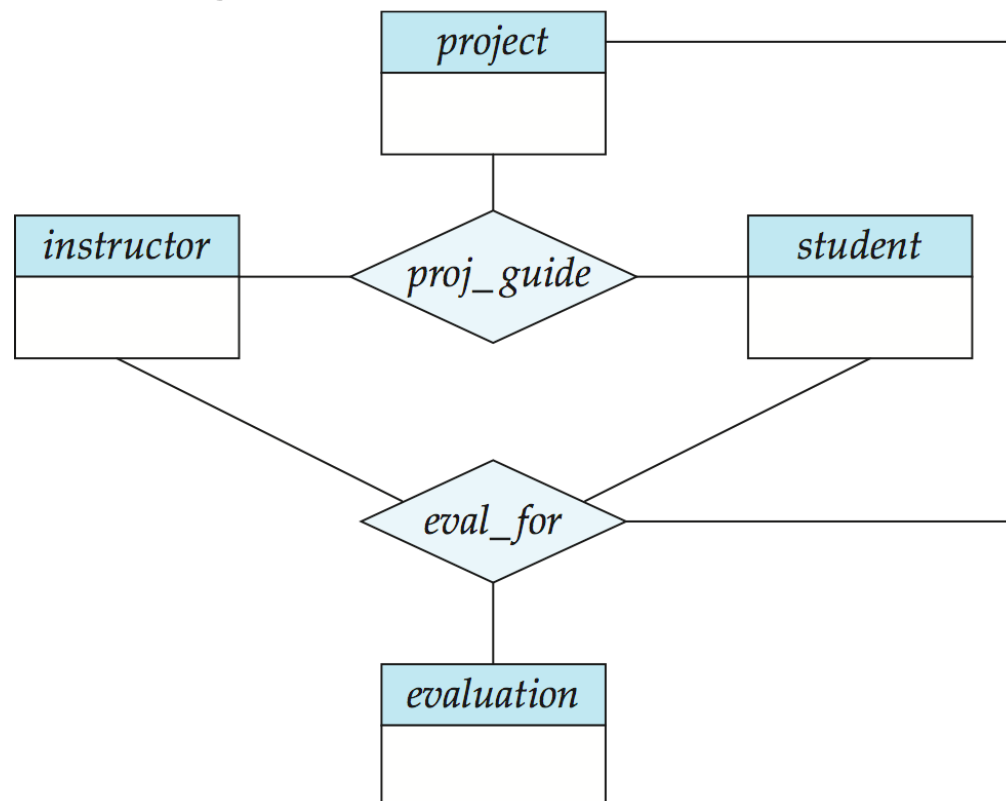


Fig.7.0.11 disjoint and overlapping in generalization

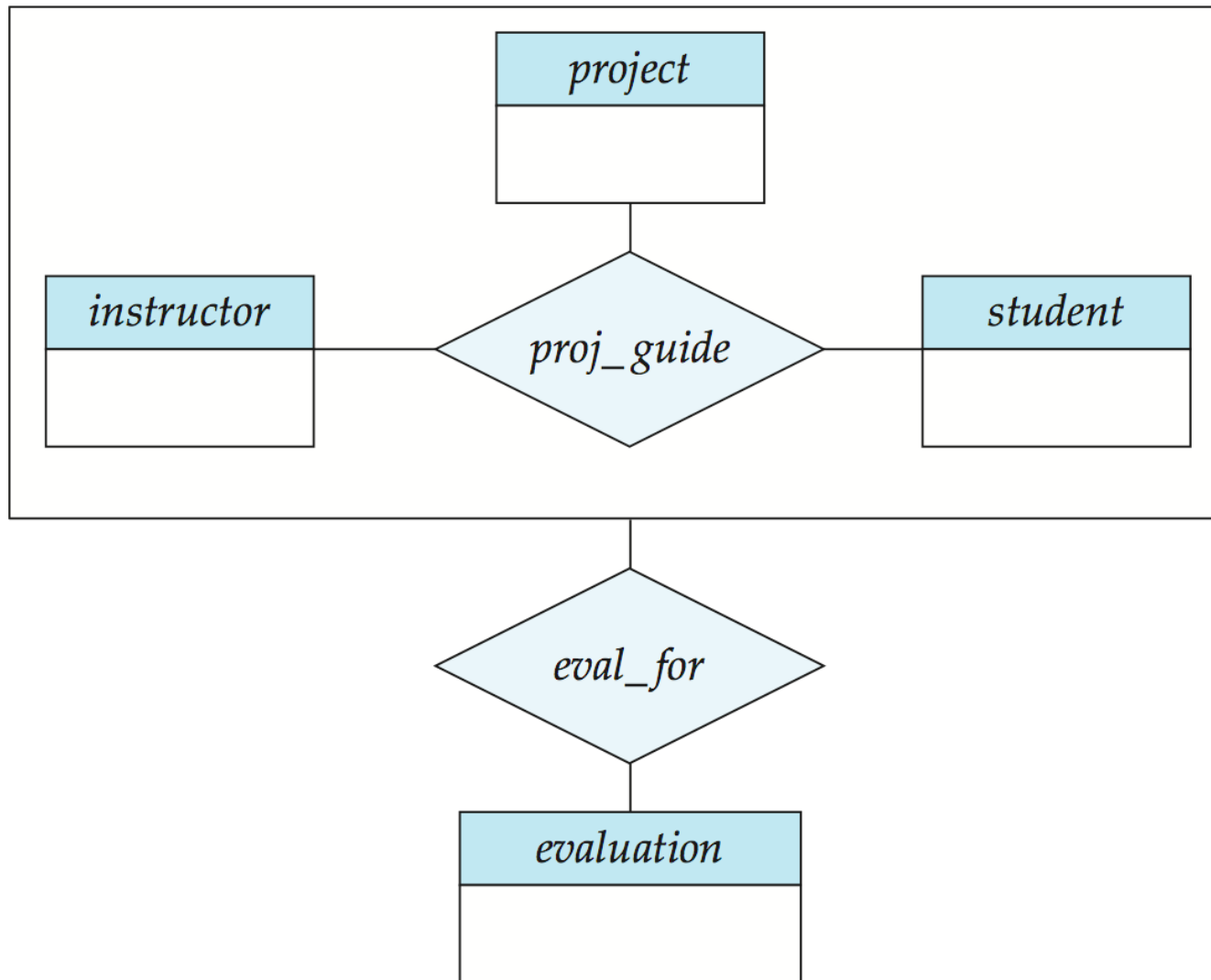
7.8.5 Aggregation

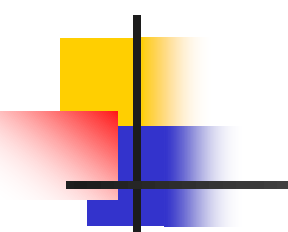
Aggregation

- to express *relationship among relationships*
- an abstraction (***methodology***) through which relationships are treated as high-level entities



Aggregation (Cont.)





Examples

- A database used in an ordering (订货, 订购) system, which contains information about *customers*, *products* and *orders*
- Two steps in DB design
 - E-R modeling
 - table reduction
- For more details about this example, refer to *Appendix D E-R Modeling and Table Reductions in Ordering Database* 

7.6 Reduction to Relational Schemas

- *Table = relation*
- Two steps in logical database design
 - generating of the *initial* relational schema from the E-R diagram, as illustrated in § 7.6
 - *relational schema normalization*, in Chapter 8
- Converting an E-R diagram to a table format is the basis for deriving a relational database design from an E-R diagram

Principles

- An *E-R diagram* describing a database can be represented by a collection of *relational tables*
- For each *entity set* and *relationship set*, there is a unique table which is assigned the name of the corresponding entity set or relationship set
- Each table has a number of columns; the columns generally corresponding to entities' or relationships' attributes, and have unique names

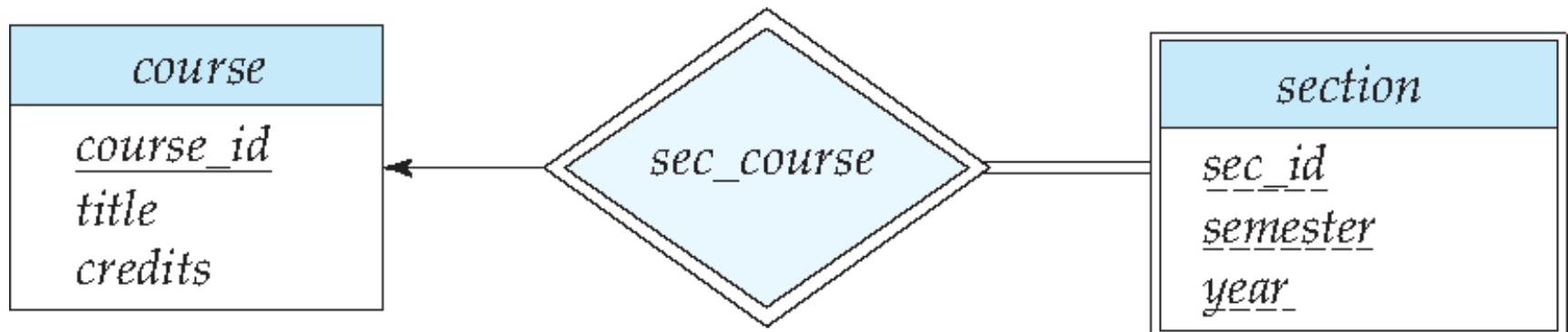
7.6.1/3 Representation of Strong/Weak Entity Sets

- A strong entity set reduces to a schema with the same attributes

student(ID, name, tot_cred)

- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set

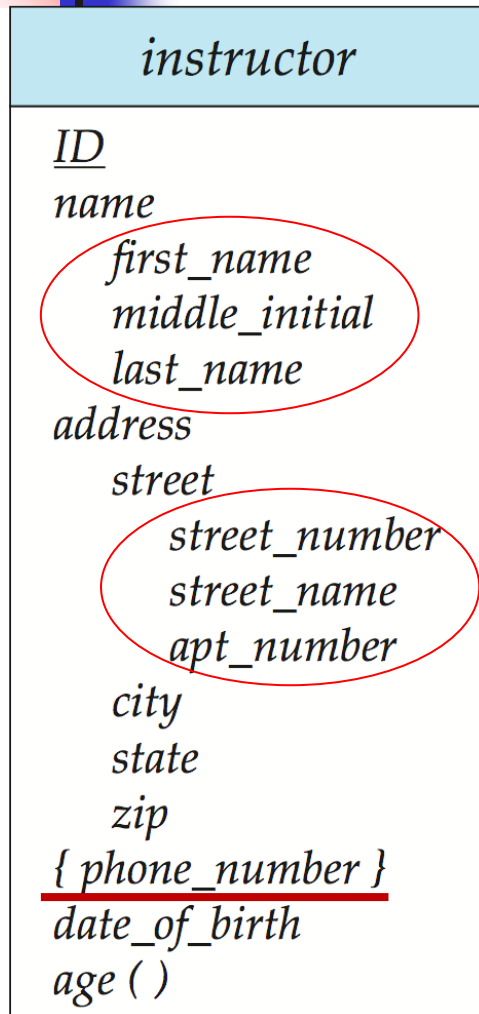
section (course_id, sec_id, sem, year)



Representation of Weak Entity Sets

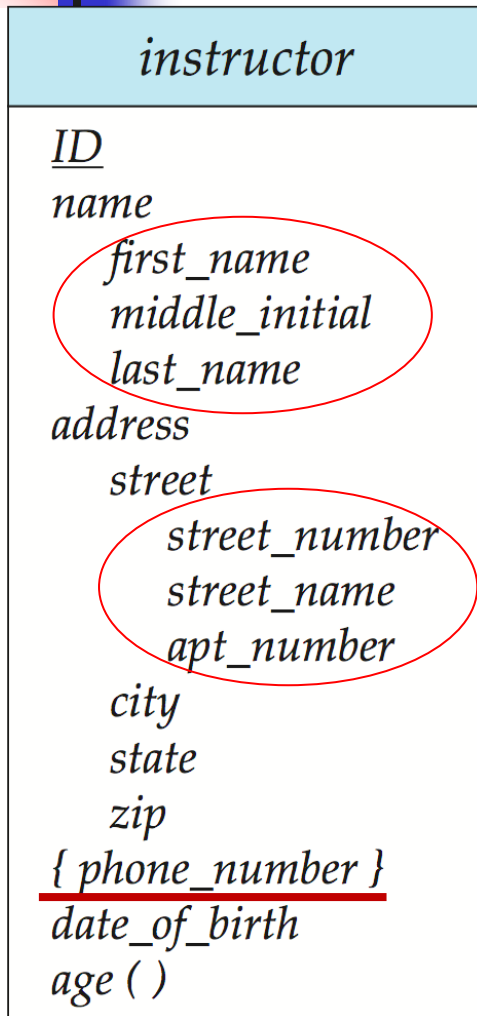
- For a weak entity set $A(a_1, a_2, \dots, a_m)$, its owner entity set is B , primary-key (B) is $b_1, b_2, \dots, b_n$, then
 - **table A** with one column for each attribute in $\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$
 - **the primary key of table A** is the union of *discriminator/partial key* of weak entity set A and primary-key(B)

7.6.2 Representation of Entity Sets with Composite Attributes



- **Composite attributes (复合属性)** are flattened out by creating a separate attribute for each component attribute
- E.g. Given entity set *instructor* with composite attribute *name* with component attributes *first_name*, *middle_initial* and *last_name*, the schema corresponding to the entity set has three attributes:
 - *name_first_name*, *middle_initial* and *name_last_name*
 - Prefix omitted if there is no ambiguity (*name_first_name* could be *first_name*)

7.6.2 Representation of Entity Sets with Composite Attributes



- Ignoring multivalued attributes, extended instructor schema is
 - *instructor*(*ID*,
 first_name, *middle_initial*, *last_name*,
 street_number, *street_name*,
 apt_number, *city*, *state*, *zip_code*,
 date_of_birth)

Representation of Entity Sets with Multivalued Attributes

- A multivalued attribute M of an entity E is represented by a separate schema EM
- Schema EM has attributes corresponding to the primary key of E and an attribute corresponding to multivalued attribute M
- Example: Multivalued attribute $phone_number$ of $instructor$ is represented by a schema:
$$inst_phone = (\underline{ID}, \underline{phone_number})$$
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema EM
 - For example, an $instructor$ entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples:
 $(22222, 456-7890)$ and $(22222, 123-4567)$

Example

- For entity set *customer*(*c-id*, *c-name*, *d-names*, *street*, *city*), each entity *customer* may have several dependents, so *d-names* (i.e., *dependent-name*) is a multivalued attribute

<u><i>c-id</i></u>	<i>c-name</i>	<i>d-names</i>	<i>street</i>	<i>city</i>
321-12-2	John	{Hayes, Adams}	North	Rye
322-10-4	Smith	{Mary, Elice}	Spring	Princeton

Fig.6.0.18 Entity set *customer* with *multivalued* attribute

- the entity set *customer* is reduced into two tables
 - *c-table*, which has no multivalued attribute *d-names*

<u><i>c-id</i></u>	<i>c-name</i>	<i>street</i>	<i>city</i>
321-12-2	John	North	Rye
322-10-4	Smith	Spring	Princeton

- *d-table*, which has primary key {*c-id*, *d-names*}, and is as follows

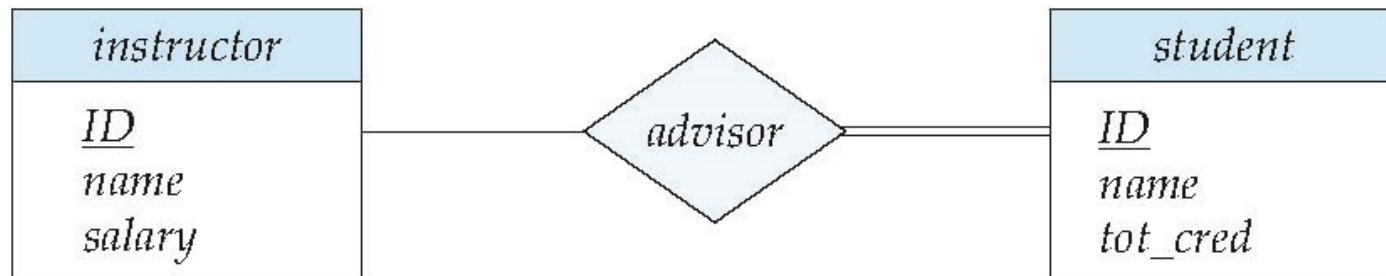
Reduction of Multivalued Attributes (cont.)

<u><i>c-id</i></u>	<u><i>d-names</i></u>
321-12-2	Hayes
321-12-2	Adams
322-10-4	Mary
322-10-4	Elice

7.6.4 Representation of Relationship Sets

- Reduction of a relationship set into tables are *strongly* dependent on the *mapping cardinality* constraint and *total/partial constraints* related to this relationship set
- A *many-to-many* relationship set is represented as a table with columns for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set

Example



instructor(ID, *name*, *salary*)
student(ID, *name*, *tot_cred*)
advisor(*instructor.ID*, *student.ID*)

Fig.8.0.14 Reduced *advisor* table

Many-to-many

- For relationship set R between entity set A and B , primary-key(A) $\cup$ primary-key(B) = $\{a_1, a_2, \dots, a_m\}$, descriptive attributes of R are $b_1, b_2, \dots, b_n$, then R can be reduced to
 - table R with one column for each attribute in $\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$

Many-to-one/One-to-many

- Many-to-one and one-to-many relationship sets *that are total on the many-side* can be represented by adding an extra attribute to the “many” side, containing the primary key of the “one” side
- For a relationship sets R , which satisfies that (refer to Fig.7.0.15)
 - *many-to-one* mapping from entity set A to B , *and*
 - *the many-side* entity set A *totally participates in* R, then R should not be represented as an independent table, it can be reduced to the table A from entity set A , by
 - adding *the primary key of* B into table A
 - adding *descriptive attributes of* R into table A
- For a *one-to-many* relationship sets R from A to B , the similar reduction can be conducted on the basis of table B .

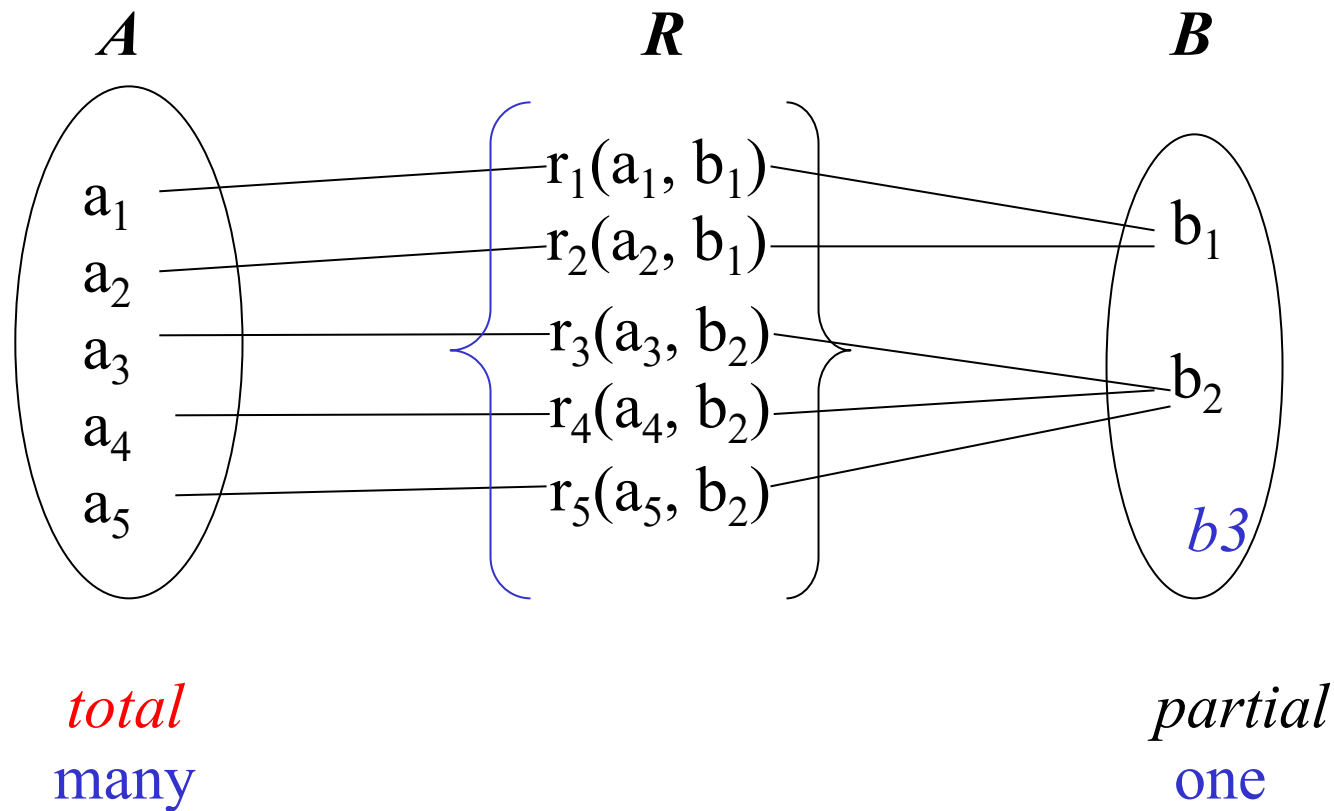
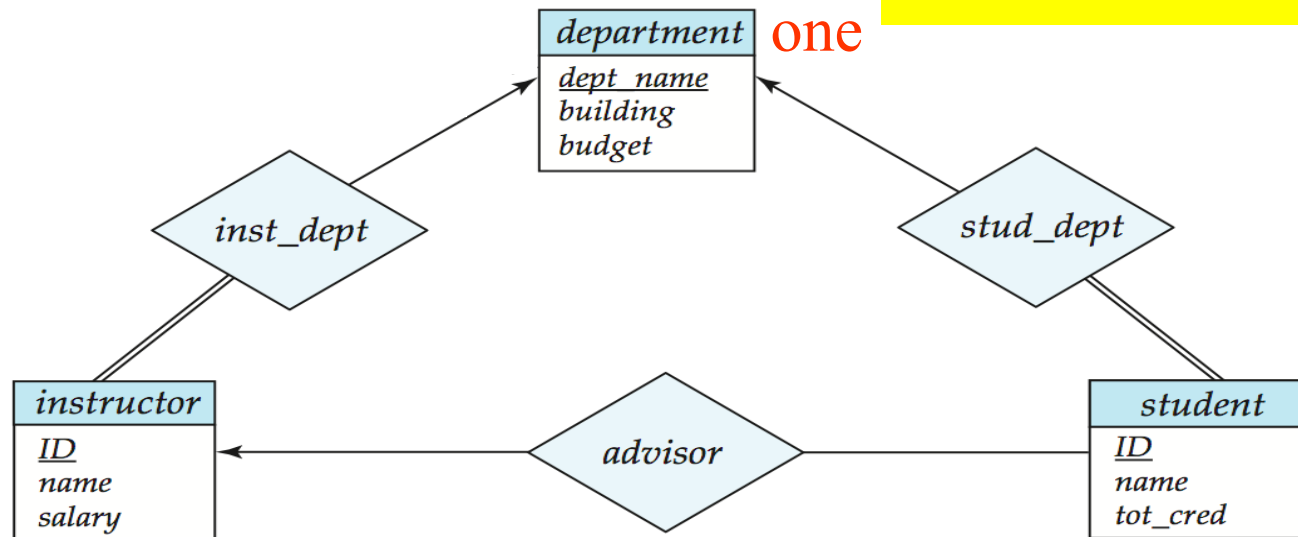


Fig. 7.0.15 A *many-to-one* relationship set R

Example

- Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from *instructor*
 - instructor*(ID, name, salary, *dept_name*)
 - department*(*dept_name*, building, budget)

foreign key referencing
constraint between
instructor and *department*



total
many

Fig.6.0.16 E-R diagram

Many-to-one Relationship Sets with *partial participation*

The relationship is reduced to a independent table

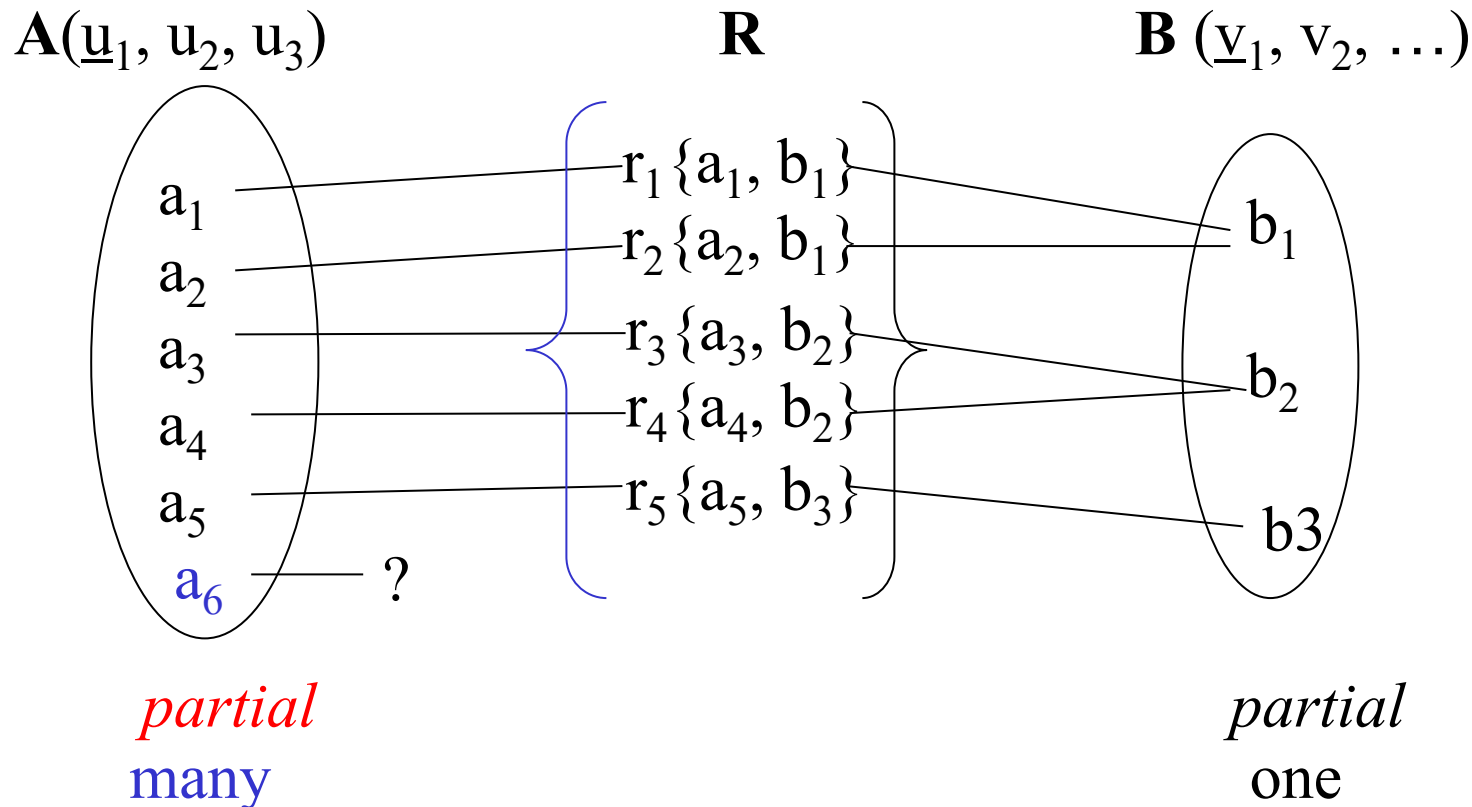


Fig.6.0.17-1 Why not reducing R into table A



在数据库系统中，尽可能避免关系表中的空值 **null**
 —用 **default value** 代替 **null**

If $R = \{r_1, r_2, r_3, r_4, r_5\}$ is reduced to table

$$A'(\underline{u_1}, u_2, u_3, v_1) = \{a_1', a_2', a_3', a_4', a_5', a_6'\},$$

- a_6 does not participate in R
- thus, a_6' will have **null** on the attribute v_1

Table $A' = \{a_1', a_2', a_3', a_4', a_5', a_6'\}$

	<u>u_1</u>	<u>u_2</u>	<u>u_3</u>	<u>v_1</u>	
a_1'	*	*	*	*	$r_1\{a_1, b_1\}$
a_2'	*	*	*	*	$r_2\{a_2, b_1\}$
a_3'	*	*	*	*	$r_3\{a_3, b_2\}$
a_4'	*	*	*	*	$r_4\{a_4, b_2\}$
a_5'	*	*	*	*	$r_5\{a_5, b_3\}$
a_6'	*	*	*	null	

Fig.7.0.17-2 Why not reducing R into table A

Reduction of *one-to-one* Relationship Sets

- **One**-to-one relationship can be viewed as a specialized instance of **many**-to-one relationship.
- For *one-to-one* relationship set R relating entity set A and B , assuming one of two entity sets, i.e. A is *totally participating* R , then
 - R is represented by adding *B 's primary key* to the table A of entity set A
 - *R 's descriptive attributes* are also added to table A

Reduction of Specialization/Generalization

■ Method 1: ▶

- Form a schema for the higher-level entity
- Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- Drawback: getting information about, an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema

Representing Specialization via Schemas

- Method 2: 
- Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

- Drawback: *name*, *street* and *city* may be stored redundantly for people who are both students and employees

- Method 1:

- for the three entity sets, creating three tables

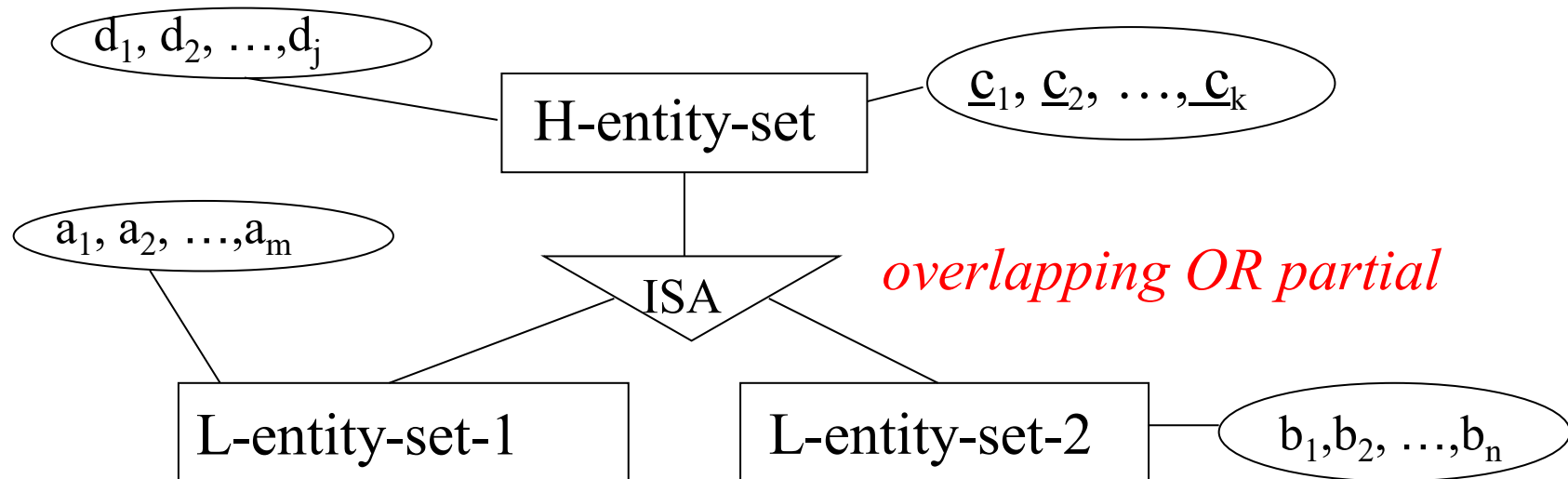


Fig.8.0.19 E-R diagram with *generalization /specialization*

Representation of Generalization (cont.)

- $T_1 (c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j)$ for H-entity-set
- $T_2 (a_1, a_2, \dots, a_m, c_1, c_2, \dots, c_k)$ for L-entity-set1
- $T_3 (b_1, b_2, \dots, b_n, c_1, c_2, \dots, c_k)$ for L-entity-set2
- note:
 - $c_1, c_2, \dots, c_k$ are **primary** attributes of H-entity-set
 - $c_1, c_2, \dots, c_k$ are the *foreign key* from T_2 and T_3 *referencing* T_1 , representing the association between H-entity-set and L-entity-set1/ L-entity-set2.

Representation of Generalization (cont.)

- Method 2. if generalization is *disjoint* and *complete*, creating two tables

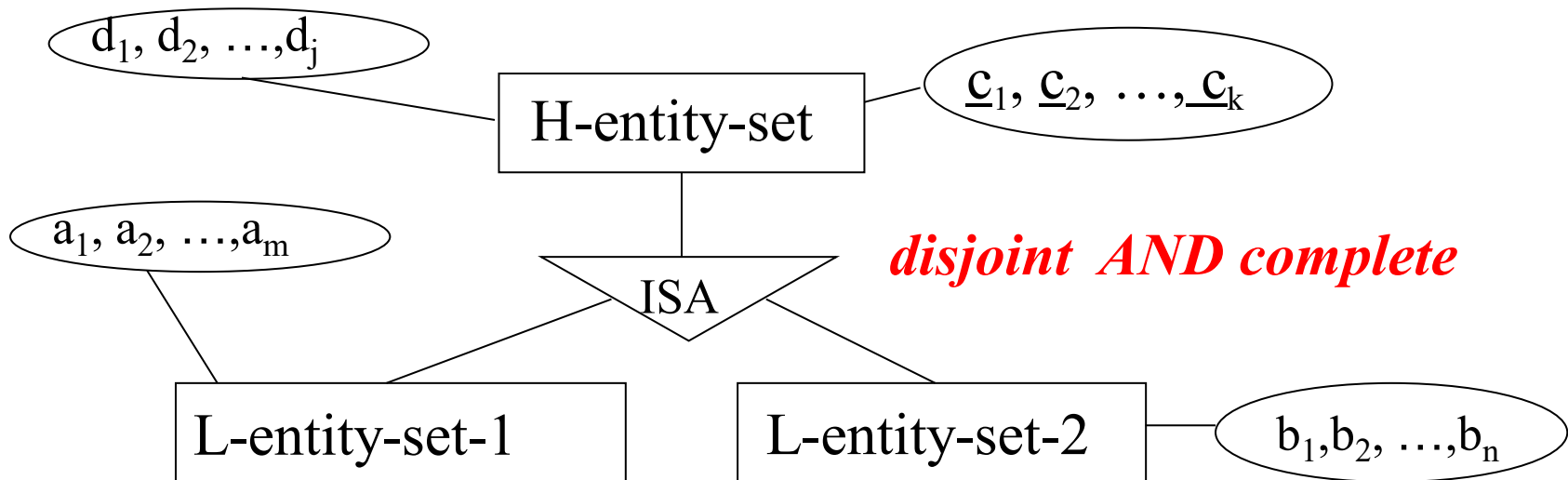
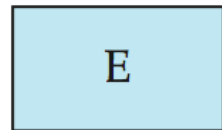


Fig.8.0.20 E-R diagram with *generalization /specialization* (II)

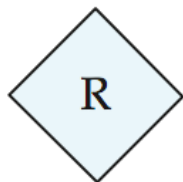
Representation of Generalization (cont.)

- $T_2(a_1, a_2, \dots, a_m, c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j)$ for L-entity-set1
- $T_3(b_1, b_2, \dots, b_n, c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j)$ for L-entity-set2
- $a_1, a_2, \dots, a_m$ are attributes of L-entity-set1, $b_1, b_2, \dots, b_n$ are attributes of L-entity-set2, and $c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j$ are **all attributes** of H-entity-set
- *replacing* high-level entity set H-entity-set *with* T_2 and T_3 of lower-level entity sets.

7.9 Symbols Used in E-R Notation 略



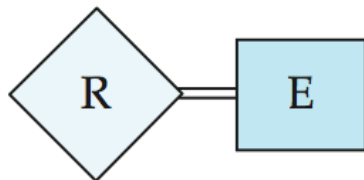
entity set



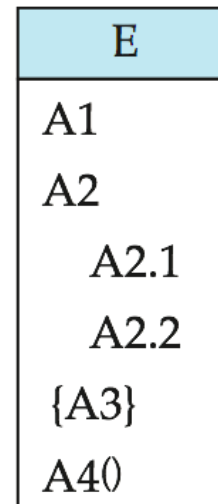
relationship set



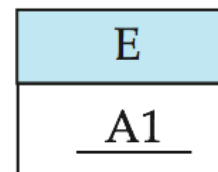
identifying
relationship set
for weak entity set



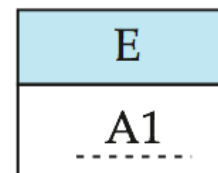
total participation
of entity set in
relationship



attributes:
simple (A1),
composite (A2) and
multivalued (A3)
derived (A4)

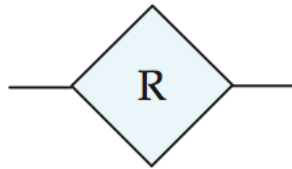


primary key

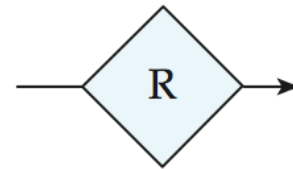


discriminating
attribute of
weak entity set

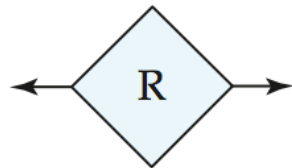
Symbols Used in E-R Notation (Cont.)



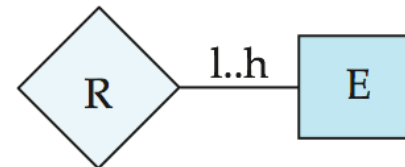
many-to-many
relationship



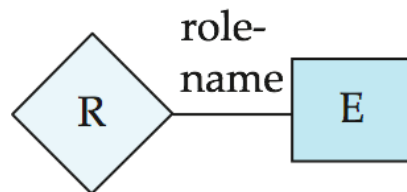
many-to-one
relationship



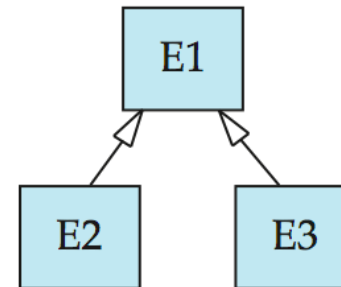
one-to-one
relationship



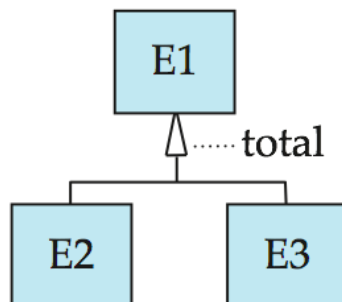
cardinality
limits



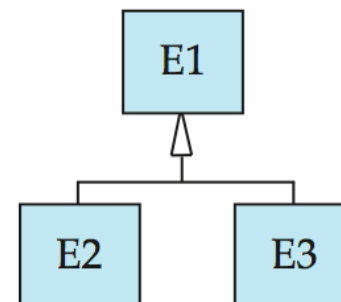
role indicator



ISA: generalization
or specialization



total (disjoint)
generalization

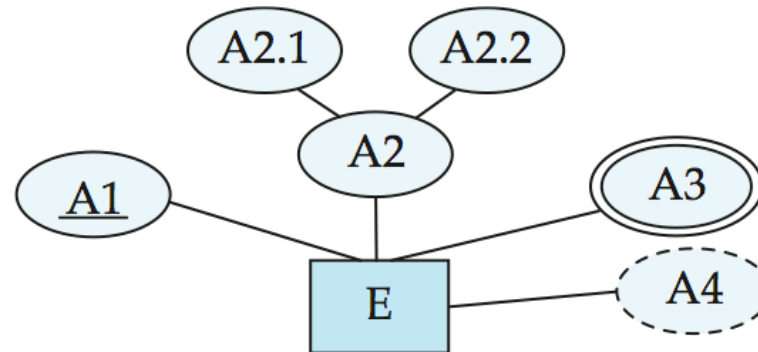


disjoint
generalization

Alternative ER Notations

- Chen, IDE1FX, ...

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



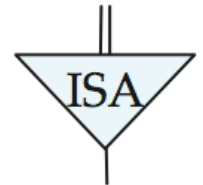
weak entity set



generalization



total
generalization



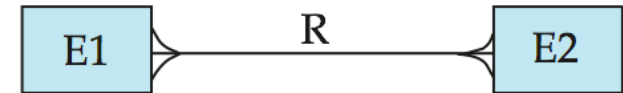
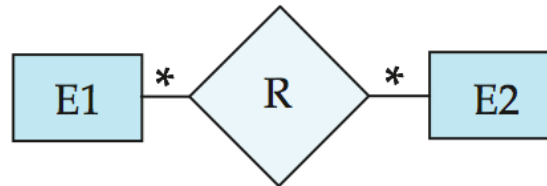
Alternative ER Notations

feet notation)

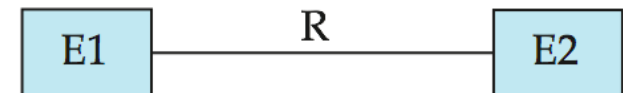
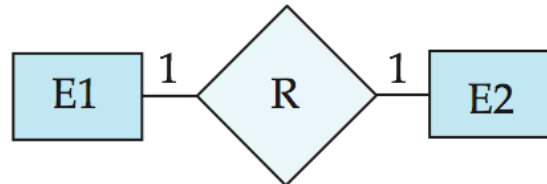
Chen

IDE1FX (Crows

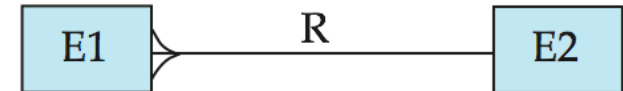
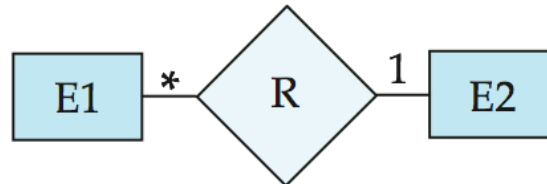
many-to-many
relationship



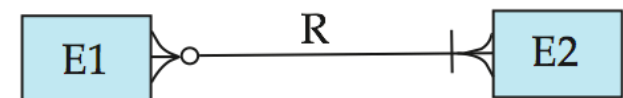
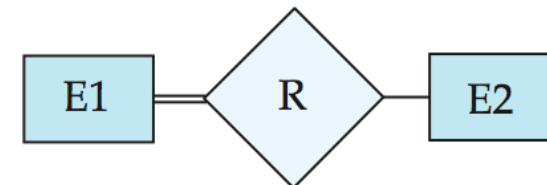
one-to-one
relationship

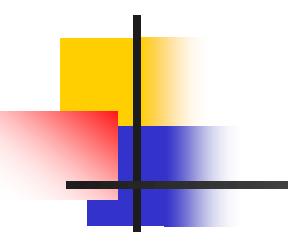


many-to-one
relationship



participation
in R: total (E1)
and partial (E2)



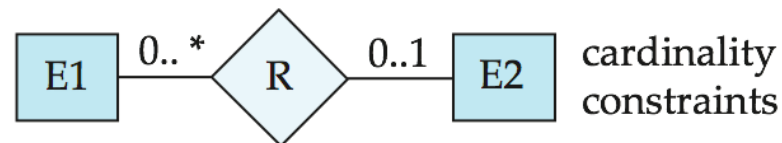
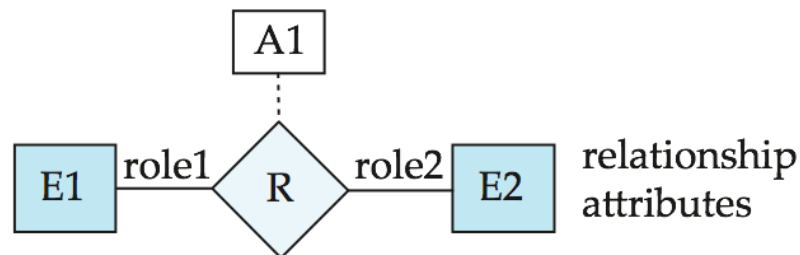
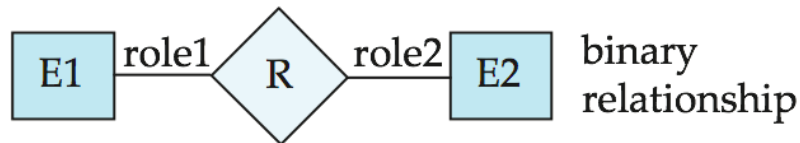
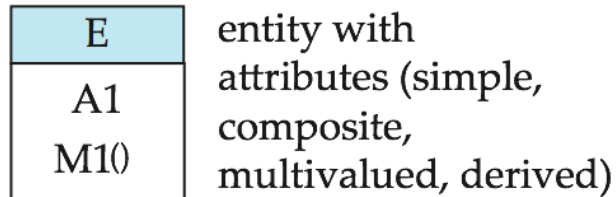


UML

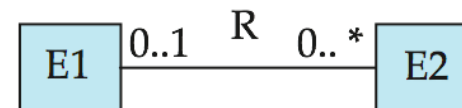
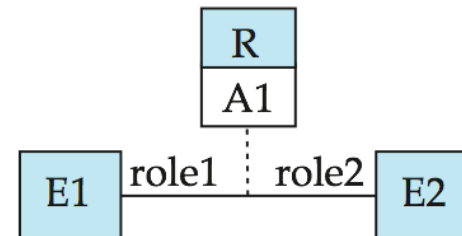
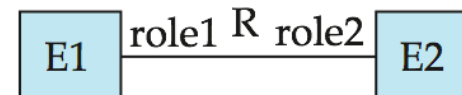
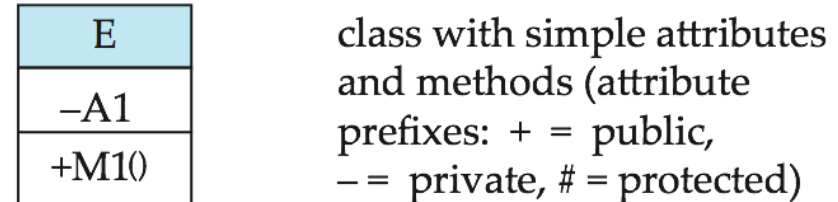
- **UML**: Unified Modeling Language
- UML has many components to graphically model different aspects of an entire software system
- UML Class Diagrams correspond to E-R Diagram, but several differences.

ER vs. UML Class Diagrams

ER Diagram Notation



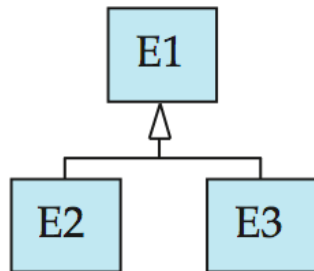
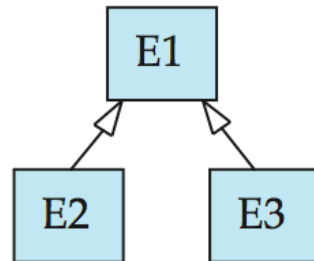
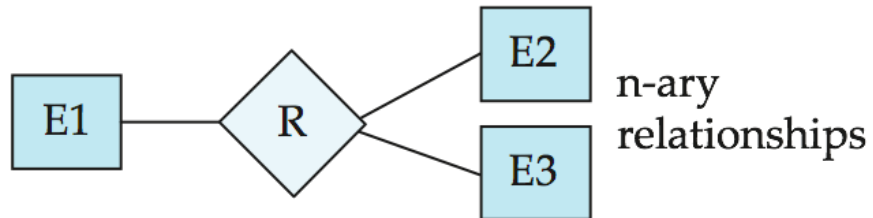
Equivalent in UML



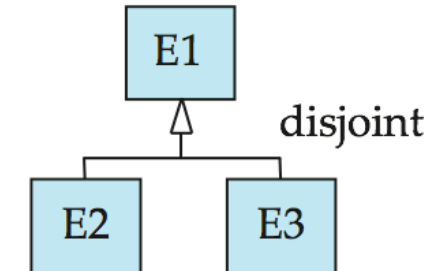
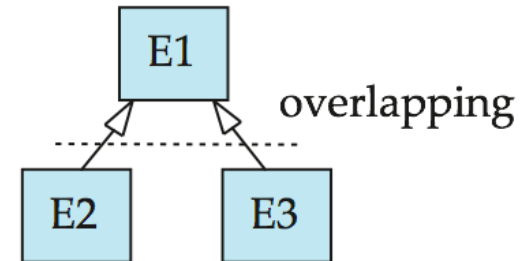
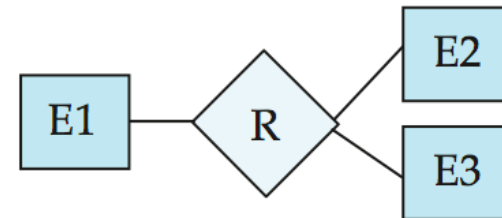
*Note reversal of position in cardinality constraint depiction

ER vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML



*Generalization can use merged or separate arrows independent of disjoint/overlapping

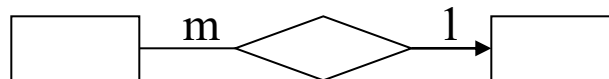
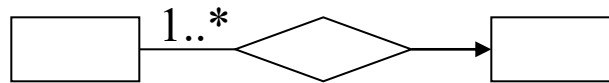
UML Class Diagrams (Cont.)

- Binary relationship sets are represented in UML by just drawing a line connecting the entity sets. The relationship set name is written adjacent to the line.
- The role played by an entity set in a relationship set may also be specified by writing the role name on the line, adjacent to the entity set.
- The relationship set name may alternatively be written in a box, along with attributes of the relationship set, and the box is connected, using a dotted line, to the line depicting the relationship set.

注意

- 画E-R图时，mapping cardinality、participation和cardinality limits约束的画法必须统一，不允许混用

- 反例



Schema Diagram in Relational Model

SQL Server生成的模式图

DESKTOP-ST5K5AP\SQLEXPRESS.University - Diagram_0* - Microsoft SQL Server Management Studio

文件(F) 编辑(E) 视图(V) 项目(P) 调试(D) 表设计器(L) 数据库关系图(M) 工具(T) 窗口(W) 帮助(H)

新建查询(N) 100%

象资源管理器

连接 数据库

DESKTOP-ST5K5AP\SQLEXPRESS (SQL

数据库

系统数据库

University

数据库关系图

表

系统表

FileTables

dbo.advisor

dbo.classroom

dbo.course

dbo.department

dbo.instructor

dbo.prereq

dbo.section

dbo.student

dbo.teaches

dbo.takes

dbo.timeslot

视图

同义词

可编程性

Service Broker

存储

安全性

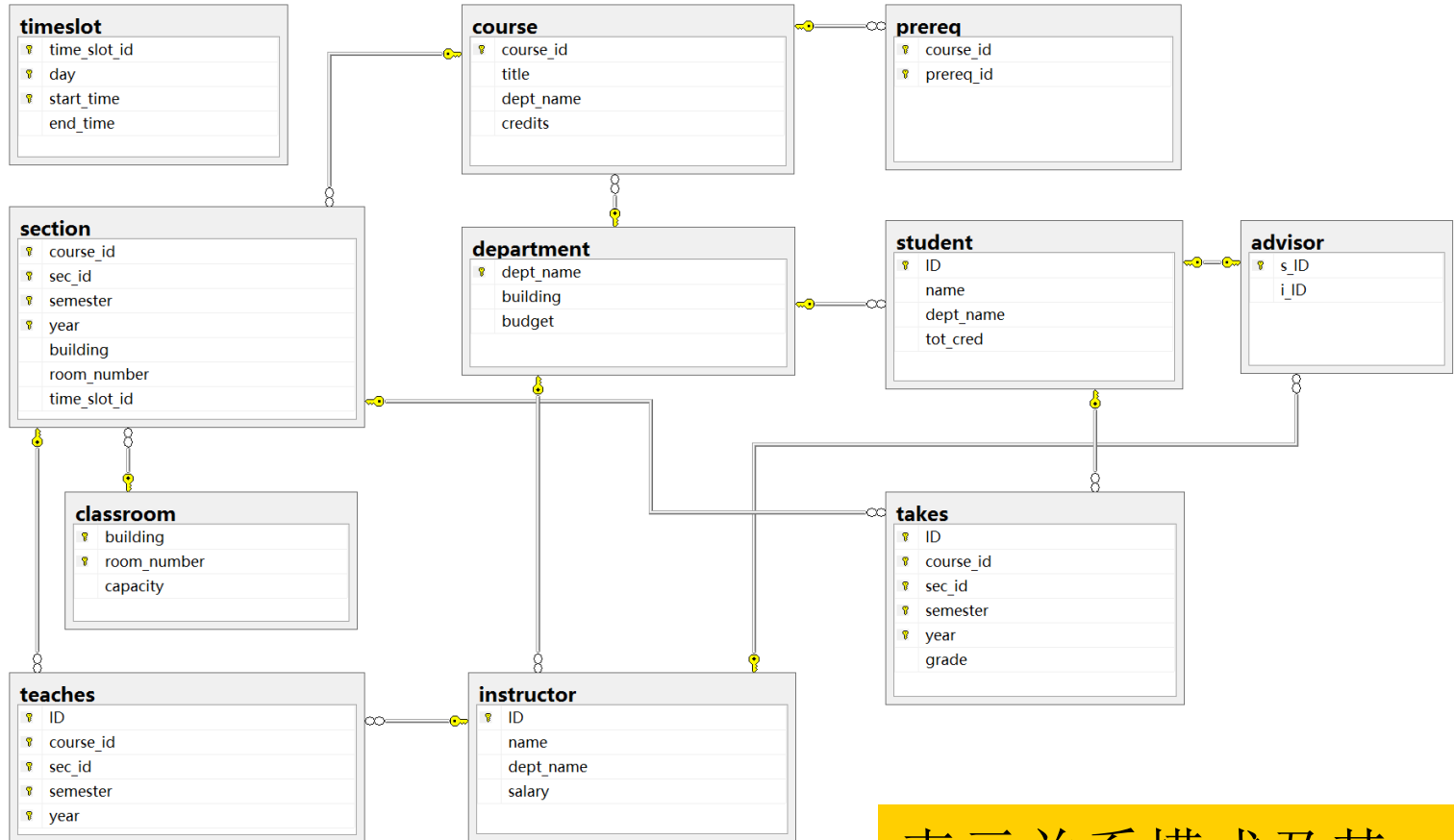
安全性

服务器对象

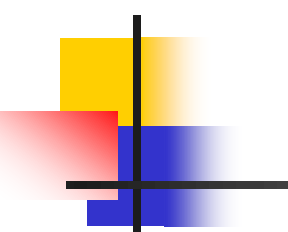
复制

管理

DESKTOP-ST5K5AP...ity - Diagram_0* x



表示关系模式及其相互间的外键关联关系，不是E-R图！！

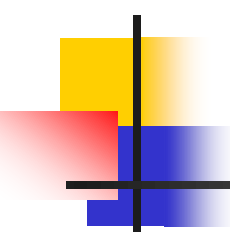


Excises

- 1. 将教材中的习题7.20中的E-R图（图7-29）转换为关系模式。
- 2. 将下页PPT中的数据需求用E-R图进行描述。

Consider the following information about the pharmacies, patients and drugs  北京邮电大学

- (1) Patients are identified by an SSN, and their names, addresses, and ages must be recorded.
- (2) Doctors are identified by an SSN. For each doctor, the name, specialty, and years of experience must be recorded.
- (3) Each pharmaceutical company (制药公司) is identified by name and has a phone number.
- (4) For each drug, the trade name and formula (成份) must be recorded. Each drug is produced by a given pharmaceutical company, and the trade name identifies a drug uniquely from among the products of that company.
- (5) Each pharmacy(药房) has a name, address, and phone number. Each pharmacy is identified by ID.
- (6) Every patient has a primary doctor. Every doctor has at least one patient.
- (7) Each pharmacy sells several drugs and has a price for each. A drug could be sold at several pharmacies, and the price could vary from one pharmacy to another.
- (8) Doctors prescribe drugs for patients. A doctor could prescribe one or more drugs for several patients, and a patient could obtain prescriptions from several doctors. Each prescription has a date and a quantity associated with it.
- (9) Pharmaceutical companies have long term contracts with pharmacies. A pharmaceutical company can contract with several pharmacies, and a pharmacy can contract with several pharmaceutical companies. For each contract, you have to store a start date, an end date.



Excises

- 6.2, 6.6: 画出E-R图并转换成关系模式

Appendix D E-R Modeling and Table Reductions in Ordering Database

- A database used in an ordering (订货/订购) system contains information about *customers*, *products* and *orders*. The following information is to be included:
 - for each ***customer***
 - *customer-number*, *ship-to-addresses* (several per customer), *name*, *balance*, *discount*
 - for each ***order*** (订单)
 - *order number*, *customer number*, *ship-to address* (one per order), *date of ordering* (including *year*, *month*, and *day*), *products* (several kinds per order), and *quantity ordered of each product*

Table Reductions in Ordering Database (cont.)

- for each kind of *product*
 - *product-number, name, manufacturing-plant, quantity on hand*

- Entity sets: *Customer, Order, Product* and their *attributes*.

In *Customer*, *ship-to addresses* is a multi-valued attribute, but in *Order*, *ship-to addresses* is a single-valued attribute.

In *Order*, *date* is a composite attribute

- Some constraints(i.e., integrity constraints) about this ordering system are as follows
 - a customer may *make* one or more orders, and some customers may make no orders
 - each order must be for one (and only one) customer

Appendix D E-R Modeling and Table Reductions in Ordering Database (cont.)

- *relationship sets*
 - relationship set *make* between *Customer* and *Order*
 - *Customer* partly participate *make*, and *Order* totally participate *make*
 - in relationship set *make*, mapping cardinality from *Customer* to *Order* is *one-to-many*
- each order must *contain* at least one kind of products
- a kind of product may appear in several orders, but some kinds of products may not be ordered by any customers
- *relationship set*
 - relationship set *contain* between *Order* and *Product*, and

Table Reductions in Ordering Database (cont.)

descriptive attributes *quantity of ordered* for
relationship set *contain*

- *Product* partly participate *contain*, and *Order* totally participate *contain*
- In relationship set *contain*, mapping cardinality from *Order* to *Product* is many-to-many
- E-R diagram of the ordering DB is shown in Fig. C.1

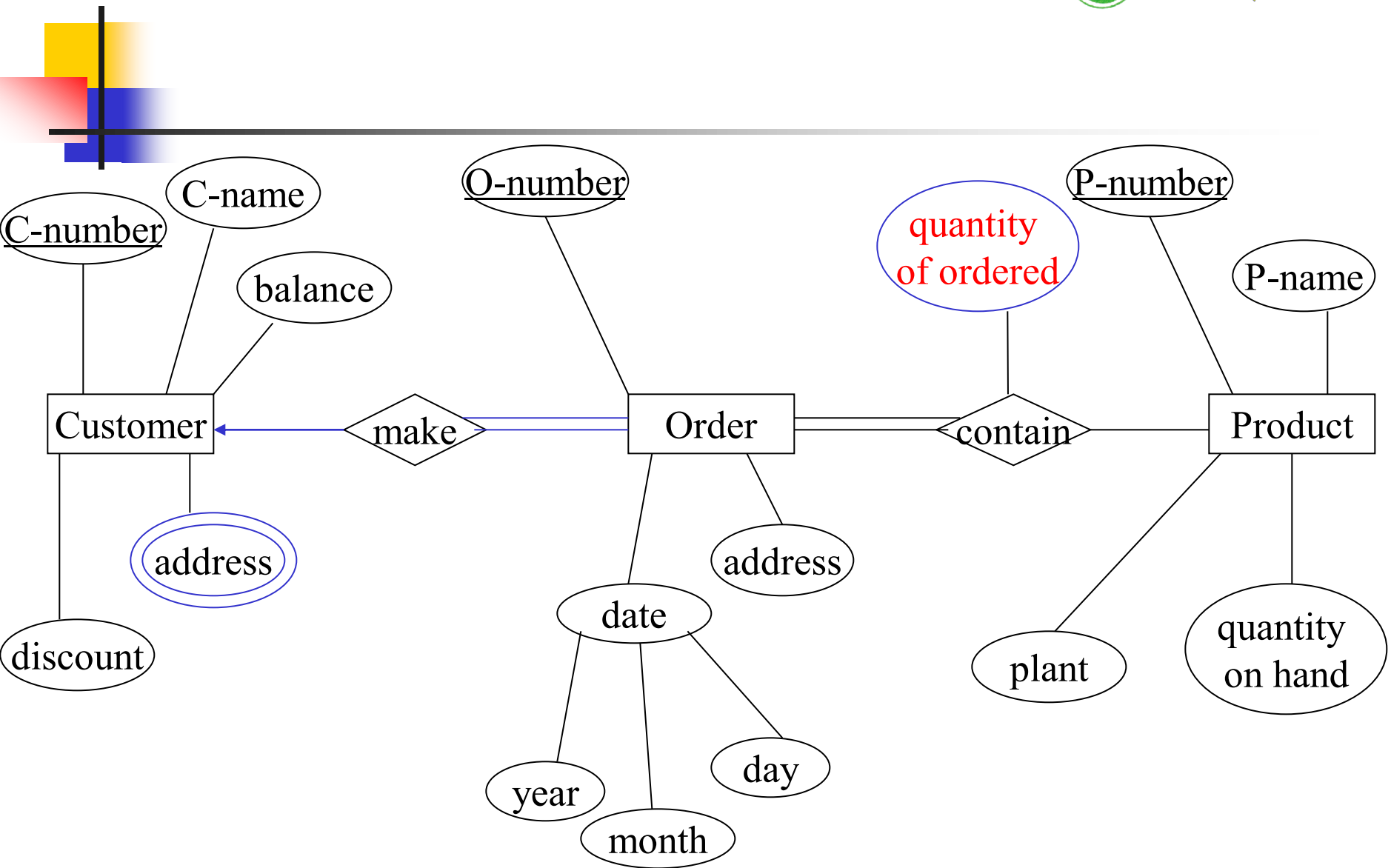


Fig. C.1. E-R diagram of the ordering DB is shown in

Table Reductions in Ordering Database (cont.)

- The E-R diagram in Fig. C.1 is then reduced to some relational tables, according to the methodologies mentioned in § 6.9
- For entity set **Customer** and its multi-valued attribute *ship-to addresses*, there are two tables
 - **customer**=(C-number, c-name, discount, balance)
 - **ship-to-addresses** = (C-number, ship-to-addresses)
- For entity set **Order** and its composite attribute **date** which should be decomposed, there is a table
 - **order1**=(O-number, ship-to-address, year, month, day)

Table Reductions in Ordering Database (cont.)

- For entity set ***Product***, there is a table
 - ***product***=(P-number, p-name, plant, quantity-on-hand)
- For relationship set ***make***, there are no descriptive attributes, mapping cardinality from ***Customer*** to ***Order*** is **one-to-many**, and ***Order*** totally participate ***make***. So, ***make*** should not be reduced into a separate table, and can be represented by adding the primary key C-number of ***Customer***, that is *C-number* into the table ***order***
 - ***order2***=(O-number, C-number, ship-to-address, year, month, day)
 - note: the primary key C-number is already in the table ***order***

Table Reductions in Ordering Database (cont.)

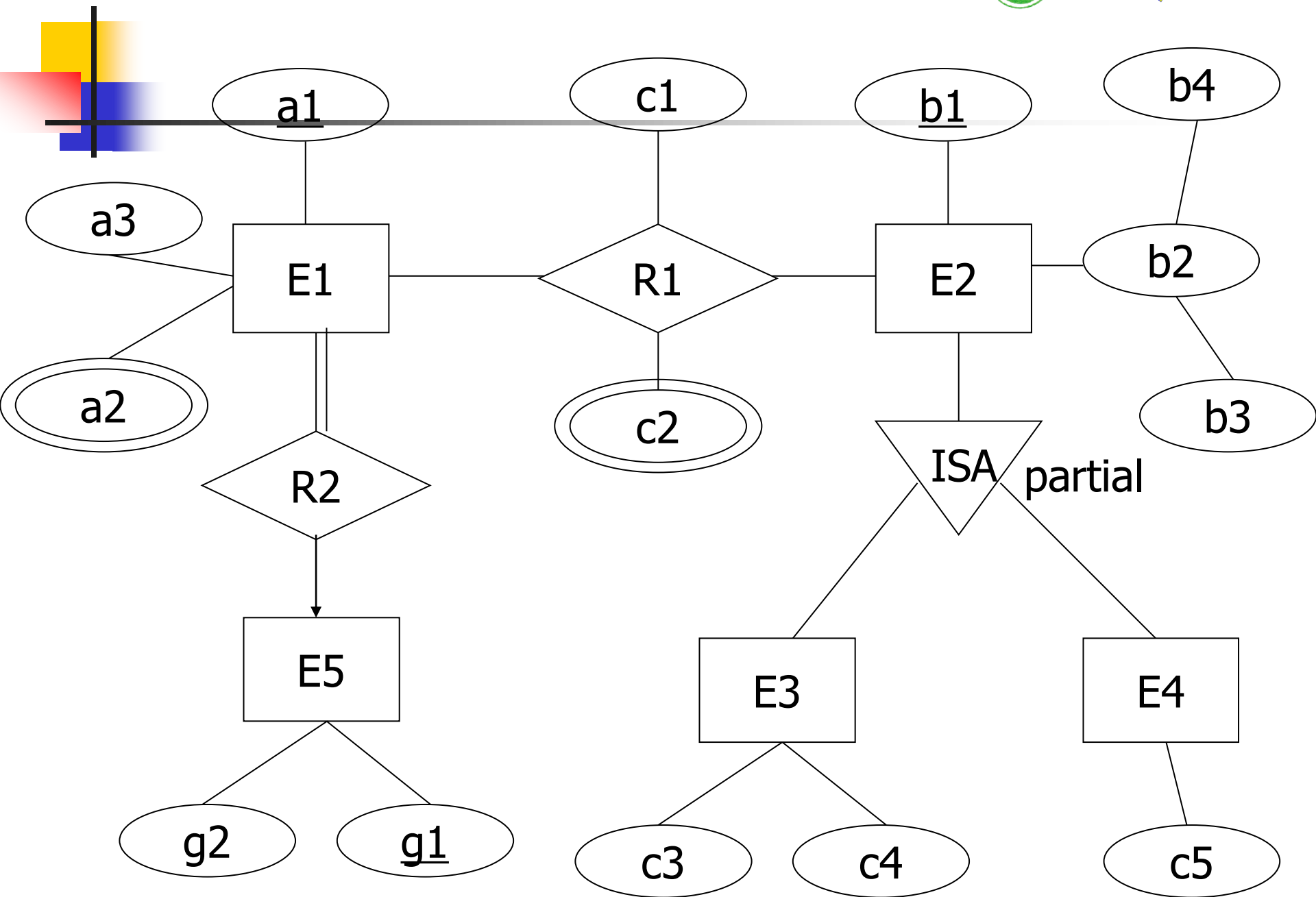
- Supposing mapping cardinality from Order to Product is many-to-many, for relationship set *contain*, there is a descriptive attribute *quantity of ordered*, *contain* should be represented as a separate table with columns for the primary keys of the two participating entity and descriptive attribute
 - *contain*=(O-number, P-number, quantity of ordered))
- Supposing mapping cardinality from Order to Product is one-to-many, because *Product* at the many side only **partly participate** relationship set *contain* , *contain* should also be represented as a separate table
 - *contain*=(O-number, P-number, quantity of ordered))

Appendix D E-R Modeling and Table Reductions in Ordering Database (cont.)

- So, the E-R diagram in Fig. C.1 can be reduced into the following five tables
 - ***customer***=(C-number, c-name, discount, balance)
 - ***ship-to-addresses*** = (C-number, ship-to-addresses)
 - ***order***=(O-number, C-number, ship-to-address, year, month, day)
 - ***product***=(P-number, p-name, plant, quantity-on-hand)
 - ***contain***=(O-number, P-number, quantity of ordered))

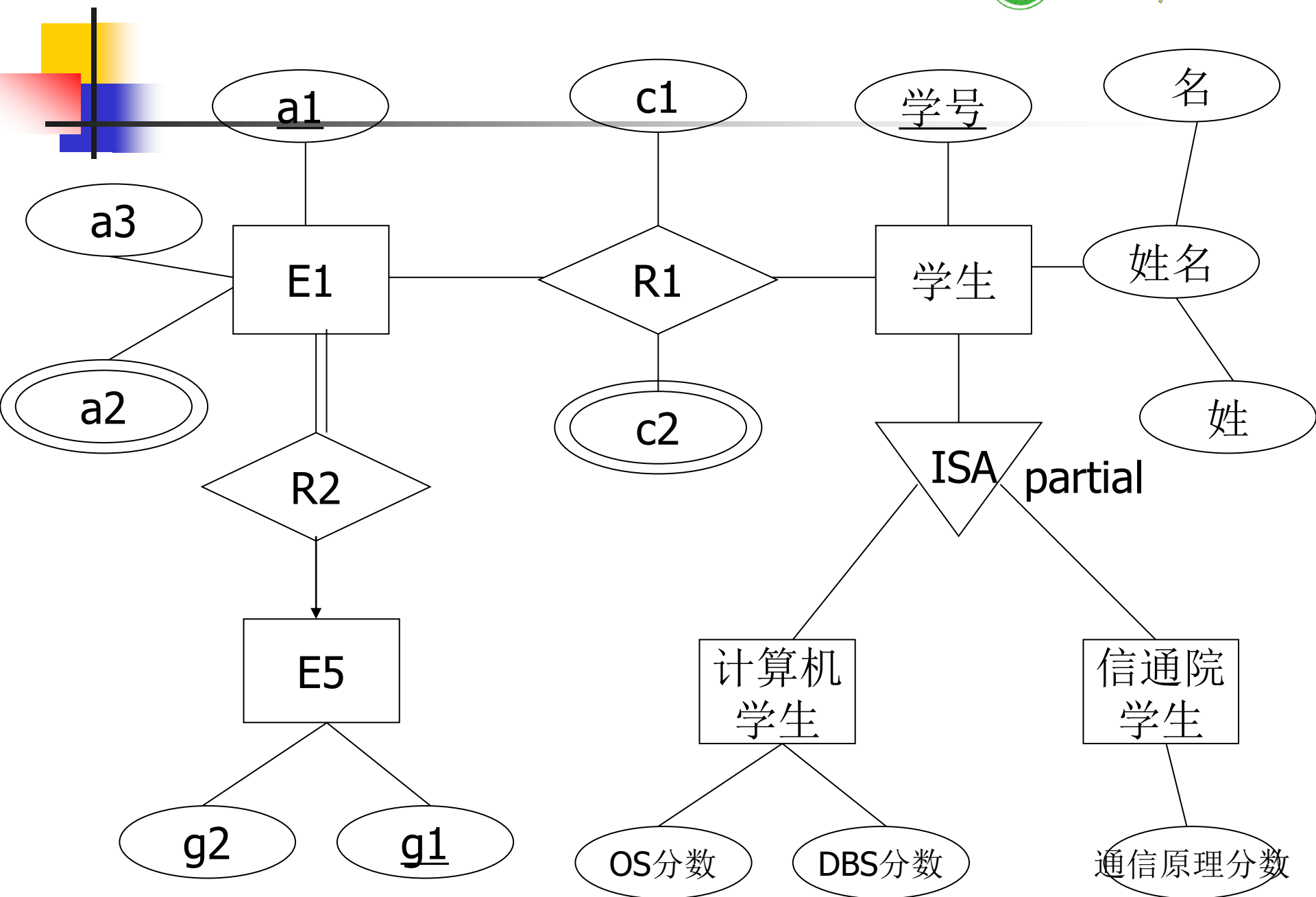
Appendix E Example

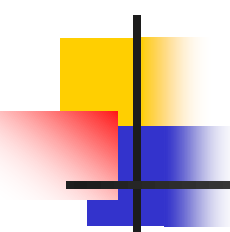
- Convert the following E-R diagram to the proper relation schemas and identify the primary key of each relation



Appendix E Example (cont.)

- $E1(\underline{a1}, a3, g1), E12(\underline{a1}, \underline{a2})$
- $E2(\underline{b1}, b3, b4)$
- $R1(\underline{a1}, \underline{b1}, c1), R12(\underline{a1}, \underline{b1}, c2)$
- $E3(\underline{b1}, c3, c4), E4(b1, c5)$
- $E5(\underline{g1}, g2)$





Have a break
